

RAG Evaluation & Testing in Production (Offline + Online)

Interview Preparation Playbook for Model Evaluators, AI QA, and SDETs for LLMs



Lamhot Siagian

RAG Evaluation & Testing in Production (Offline + Online)

Interview Preparation Playbook for Model Evaluators, AI QA, and SDETs
for LLMs

Lamhot Siagian

@growth.ideology

<https://beacons.ai/lamhotsiagian>

February 06, 2026

@growth.ideology Helping 130,000+ people improve their AI skills

How to Use This Book

This book is an **interview preparation** playbook for building and defending a production-grade evaluation strategy for Retrieval-Augmented Generation (RAG). It's written for roles like **Model Evaluator**, **AI Quality**, and **SDET for LLMs**—people who are expected to ship reliable evaluation systems, stop regressions in CI/CD, and protect users during production rollouts.

What interviewers are really testing

They want evidence that you can:

- turn fuzzy quality into **measurable signals** (metrics + rubrics),
- build **repeatable tests** (golden sets, snapshots, flake control),
- debug failures fast (tracing, diffing, root-cause isolation),
- run safe rollouts (canary, thresholds, rollback policy),
- handle drift (data, embeddings, index rebuilds, query distribution).

Study plans

- **7 days:** read Chapters 1–6, then Chapters 10–12, then Capstone.
- **14 days:** add Chapters 7–9 + calibration + judge reliability drills.
- **30 days:** implement the full Python harness and publish the repo.

Contents

1	Component-Wise RAG Evaluation	1
1.1	Core Concepts	1
1.1.1	Pipeline anatomy	1
1.1.2	Failure taxonomy	1
1.1.3	Controlled diffs	1
1.2	Decision Framework	2
1.3	Debug Playbook	2
1.4	Mini-Lab	3
1.5	Python Reference Implementation	3
1.6	Interview Q&A (15)	3
1.7	Cheat Sheet	6
2	Offline vs Online Evaluation Strategy	7
2.1	Core Concepts	7
2.1.1	Offline evaluation	7
2.1.2	Online evaluation	7
2.1.3	Alignment	7
2.2	Decision Framework	8
2.3	Debug Playbook	8
2.4	Mini-Lab	8
2.5	Python Reference Implementation	9
2.6	Interview Q&A (15)	9
2.7	Cheat Sheet	11
3	Retrieval Metrics that Predict Quality	12
3.1	Core Concepts	12
3.1.1	Classic ranking metrics	12
3.1.2	RAG-specific twist	12
3.1.3	Bucketed reporting	12
3.2	Decision Framework	13
3.3	Debug Playbook	13
3.4	Mini-Lab	13
3.5	Python Reference Implementation	14

3.6	Interview Q&A (15)	14
3.7	Cheat Sheet	16
4	Rerankers and Hybrid Retrieval	17
4.1	Core Concepts	17
4.1.1	Candidate generation vs reranking	17
4.1.2	Win-rate	17
4.1.3	Latency budgets	17
4.2	Decision Framework	18
4.3	Debug Playbook	18
4.4	Mini-Lab	18
4.5	Python Reference Implementation	19
4.6	Interview Q&A (15)	19
4.7	Cheat Sheet	21
5	Answer Relevance Scoring	22
5.1	Core Concepts	22
5.1.1	Relevance is intent satisfaction	22
5.1.2	Partial credit	22
5.1.3	Over-refusal	22
5.2	Decision Framework	23
5.3	Debug Playbook	23
5.4	Mini-Lab	23
5.5	Python Reference Implementation	24
5.6	Interview Q&A (15)	24
5.7	Cheat Sheet	26
6	Groundedness and Citation Accuracy	27
6.1	Core Concepts	27
6.1.1	Claim-level groundedness	27
6.1.2	Citation accuracy	27
6.1.3	Abstention	27
6.2	Decision Framework	28
6.3	Debug Playbook	28
6.4	Mini-Lab	28
6.5	Python Reference Implementation	29
6.6	Interview Q&A (15)	29
6.7	Cheat Sheet	31
7	LLM-as-Judge Prompt Engineering	32
7.1	Core Concepts	32
7.1.1	Judge prompt patterns	32
7.1.2	Evidence-only scoring	32
7.1.3	Variance control	32

7.2	Decision Framework	33
7.3	Debug Playbook	33
7.4	Mini-Lab	33
7.5	Python Reference Implementation	34
7.6	Interview Q&A (15)	34
7.7	Cheat Sheet	36
8	Judge Calibration and Meta-Evaluation	37
8.1	Core Concepts	37
8.1.1	Calibration sets	37
8.1.2	Stability	37
8.1.3	Thresholds with uncertainty	37
8.2	Decision Framework	38
8.3	Debug Playbook	38
8.4	Mini-Lab	39
8.5	Python Reference Implementation	39
8.6	Interview Q&A (15)	39
8.7	Cheat Sheet	41
9	Judge Failure Modes and Attacks	42
9.1	Core Concepts	42
9.1.1	Leakage and gaming	42
9.1.2	Injection hardening	42
9.1.3	Tiered judging	42
9.2	Decision Framework	43
9.3	Debug Playbook	43
9.4	Mini-Lab	43
9.5	Python Reference Implementation	44
9.6	Interview Q&A (15)	44
9.7	Cheat Sheet	46
10	Regression Testing for RAG in CI	47
10.1	Core Concepts	47
10.1.1	Golden queries	47
10.1.2	Snapshot assertions	47
10.1.3	Flake control	47
10.2	Decision Framework	48
10.3	Debug Playbook	48
10.4	Mini-Lab	48
10.5	Python Reference Implementation	49
10.6	Interview Q&A (15)	49
10.7	Cheat Sheet	50
11	Canary Evaluation and Rollback Policy	51

11.1	Core Concepts	51
11.1.1	Shadow and canary	51
11.1.2	Rollback policy	51
11.1.3	Incident playbook	51
11.2	Decision Framework	52
11.3	Debug Playbook	52
11.4	Mini-Lab	52
11.5	Python Reference Implementation	52
11.6	Interview Q&A (15)	53
11.7	Cheat Sheet	55
12	Drift Detection for RAG	56
12.1	Core Concepts	56
12.1.1	Drift types	56
12.1.2	Signals	56
12.1.3	Auto-curation	56
12.2	Decision Framework	57
12.3	Debug Playbook	57
12.4	Mini-Lab	57
12.5	Python Reference Implementation	58
12.6	Interview Q&A (15)	58
12.7	Cheat Sheet	60
A	Templates and Schemas	61
A.1	RAG Trace Record (Minimal)	61
A.2	Golden Test Case Template	62
A.3	LLM Judge Output Schema (Minimal)	62
B	Capstone: System Design Interview	63
B.1	Prompt	63
B.2	Reference Architecture (Text Diagram)	63
B.3	What a strong answer includes	63
C	Interview Master Bank (120 Questions)	65
C.1	Retrieval \& Ranking	65
C.2	Reranking \& Hybrid Retrieval	67
C.3	Generation Relevance \& Grounding	68
C.4	LLM-as-Judge Engineering	70
C.5	Regression Testing \& CI/CD	71
C.6	Online Evals, Canary, \& Drift	72
D	Hands-On Lab Playbook	75
D.1	Lab 1: Build a Golden Set in 60 Minutes	75
D.2	Lab 2: Retrieval Evaluation with Buckets	75
D.3	Lab 3: Citation Verifier (Rules + Escalation)	75

D.4	Lab 4: Oracle-Context Debugging	76
D.5	Lab 5: LLM Judge with JSON Schema	76
D.6	Lab 6: CI Gate in GitHub Actions	76
D.7	Lab 7: Canary Evaluation Simulation	76
D.8	Lab 8: Drift Monitoring Probes	76
E	Rubrics, Scorecards, and Quality Gates	77
E.1	Relevance Rubric (1–5)	77
E.2	Groundedness Rubric (Claim-Level)	78
E.3	Citation Accuracy Rubric	78
E.4	Critical Error Checklist	78
E.5	Quality Gate Matrix (Example)	79
E.6	Threshold Setting Rules of Thumb	79
F	Production Case Studies: Regressions and Fixes	80
F.1	Case 1: Index Rebuild Improves Recall but Breaks Citations	80
F.2	Case 2: Prompt Update Increases Relevance but Reduces Groundedness	80
F.3	Case 3: Embedding Model Upgrade Causes Silent Drift	80
F.4	Case 4: Spammy Document Dominates Retrieval (Keyword Stuffing)	81
F.5	Case 5: Prompt Injection in Retrieved Evidence	81
F.6	Case 6: Canary Passed, Full Rollout Failed	81
F.7	Interview Mapping	81
G	Glossary (Practical)	82

Chapter 1

Component-Wise RAG Evaluation

Isolate the retriever, reranker, and generator so you can fix the correct knob.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

1.1 Core Concepts

1.1.1 Pipeline anatomy

A typical production RAG system has distinct stages: *ingest* → *chunk* → *index* → *retrieve* → *rerank* → *prompt-pack* → *generate* → *cite*. The QA insight: **each stage can regress independently**. Your evaluation program must isolate them.

1.1.2 Failure taxonomy

Use a stable label set to triage failures quickly and to trend regressions over time (e.g., RETRIEVAL_MISS, HALLUCINATION, CITATION_WRONG).

1.1.3 Controlled diffs

To identify root cause, change **one variable at a time**: chunker only, index only, reranker only, prompt only, model only.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

1.2 Decision Framework

Choose **which component to score first** so your metrics point to a fix (not a debate).

1. **Start from the user-risk tier:** what errors are unacceptable (unsupported claims, wrong citations, refusal failures)?
2. **Pick the evaluation target:** retriever (candidate quality), reranker (ordering), generator (intent satisfaction), citation resolver (claim-to-cite mapping).
3. **Select a stage-specific metric:** e.g., **recall@k** for candidate coverage, **MRR/nDCG** for ordering, **relevance** for intent, **groundedness** for evidence support.
4. **Plan an isolation experiment:** oracle-context (perfect evidence) vs normal retrieval; reranker on/off; prompt-pack variants.
5. **Set gates per stage:** block merges on **critical** errors; allow minor metric noise with multi-run medians.

1.3 Debug Playbook

When end-to-end quality drops, isolate the stage that changed **first**.

Checklist

1. **Confirm what changed:** model/prompt, index, embeddings, chunker, reranker, post-processing (citations).
2. **Run oracle-context:** if answers recover with gold context, the issue is retrieval/reranking; if not, it's generation/prompt.
3. **Inspect evidence flow:** top-k retrieved chunks, reranked order, truncation rate, and whether the cited chunk is actually present in the packed context.
4. **Ablate one knob:** disable reranker, revert chunker, pin corpus snapshot, then re-score.
5. **Convert to a guardrail:** add the failing case to the golden set + tag the responsible component for future triage.

1.4 Mini-Lab

Mini-Lab

Goal: Create a minimal evaluation record and failure labeling workflow.

- Capture a trace for 20 queries: retrieved chunks, scores, answer, citations, versions.
- Label each failure with a taxonomy tag.
- Produce a small failure gallery (top 5 regressions).

1.5 Python Reference Implementation

This snapshot shows a minimal component-wise evaluator: one metric for retrieval and one metric for grounding, combined into an overall score.

```

1  # Chapter 1 snapshot: component-wise RAG evaluation
2  from dataclasses import dataclass
3  from typing import List, Dict
4
5  @dataclass
6  class EvalCase:
7      question: str
8      retrieved: List[str]      # retrieved chunk IDs/texts (top-k list)
9      answer: str               # model output
10     gold_chunks: List[str]    # chunks that should contain the answer
11
12     def recall_at_k(retrieved: List[str], gold: List[str], k: int) -> float:
13         # Retrieval metric: what fraction of gold chunks appear in top-k?
14         hits = sum(1 for c in retrieved[:k] if c in gold)
15         return hits / max(1, len(gold)) # avoid division-by-zero
16
17     def component_eval(case: EvalCase) -> Dict[str, float]:
18         r = recall_at_k(case.retrieved, case.gold_chunks, k=5)
19         # Grounding proxy: does the answer include evidence from any gold chunk?
20         grounded = 1.0 if any(g in case.answer for g in case.gold_chunks) else 0.0
21         overall = (r + grounded) / 2
22         return {"retrieval_recall@5": r, "groundedness": grounded, "overall": overall}

```

Listing 1.1: Chapter 1: Component-Wise RAG Evaluation (Python snapshot)

1.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
----------	-----------------------------

Why do component-wise evals beat end-to-end-only evals?	End-to-end scores hide root cause. Component-wise evals isolate which stage regressed (retrieval, reranking, prompting, generation, citation mapping), so fixes are targeted and regression tests can be scoped per component. <i>Rubric:</i> 2 = isolation + targeted fixes + per-component regression tests.
List 6 common RAG failure modes.	Retrieval miss, wrong evidence, misleading evidence, truncation/context stuffing, hallucinated claim, wrong/missing citation. <i>Rubric:</i> 2 = includes retrieval + generation + citation failures.
What is the minimum trace you must log for debugging?	Query, retrieved chunk IDs + scores + text hashes, rerank order, final answer, citations (doc+chunk), versions (model/prompt/index/chunker/embeddings), latency and token usage. <i>Rubric:</i> 2 = includes evidence + version pins.
How do you separate retrieval quality from generation quality?	Run retrieval-only labeling (chunk relevance) and generation-only runs with oracle context (gold passages). Compare: if oracle fixes the answer, retrieval is the issue; if oracle still fails, generation/prompt is the issue. <i>Rubric:</i> 2 = oracle-context experiment.
Context relevance vs answer relevance?	Context relevance measures whether retrieved chunks match the query; answer relevance measures whether the response satisfies intent. They diverge when the model ignores evidence or overgeneralizes. <i>Rubric:</i> 2 = explains divergence.
Design an eval to detect chunking regressions.	Hold model/prompt fixed; vary chunker only; measure retrieval recall@k, context precision, groundedness, citation accuracy. Add boundary tests where a required fact spans chunk boundaries. <i>Rubric:</i> 2 = controlled diff + boundary cases.
Recall@k is good but groundedness is bad—next check?	Check reranker order, prompt truncation, and context utilization (did the model cite/use the relevant chunk). Often the right chunk is present but not surfaced early enough. <i>Rubric:</i> 2 = rerank + truncation + utilization.

Create a failure taxonomy label set.	RETRIEVAL_MISS, WRONG_EVIDENCE, EVIDENCE_CONFLICT, HALLUCINATION, CITATION_WRONG, CITATION_MISSING, FORMAT_FAIL, REFUSAL_ERROR. <i>Rubric:</i> 2 = actionable labels include citation + refusal.
How do you prevent “debugging by vibes”?	Standardize traces, label failures with taxonomy, compare against baseline diffs, and pin versions so failures are reproducible. Keep a failure gallery with top regressions. <i>Rubric:</i> 2 = reproducibility + diffing.
What golden-set tags matter most?	Risk tier, recency-sensitive, multi-hop, entity ambiguity, long-context/truncation, injection/adversarial, multi-lingual/jargon. <i>Rubric:</i> 2 = includes risk + recency + adversarial.
Design a component-wise eval pipeline.	Dataset registry → batch runner → retrieval scorer → generation scorer → judge scorer → reports + failure gallery; store artifacts and version metadata for replay. <i>Rubric:</i> 2 = stored artifacts + replay.
Build a failure gallery for engineers.	Show query, top chunks, answer, citations, score breakdown, and diff vs baseline; filter by taxonomy and bucket; link to traces. <i>Rubric:</i> 2 = diff + taxonomy filters.
Where do you put quality gates?	PR: small deterministic golden set; nightly: broad stochastic evaluation; post-deploy: canary/shadow + drift monitoring. Each stage trades speed for coverage. <i>Rubric:</i> 2 = stage separation + rationale.
Prompt injection in retrieved docs changes answers—what do you do?	Sanitize/strip instruction-like text, add injection detectors, constrain system prompt, and build an adversarial injection suite. Alert on spikes and quarantine suspicious sources. <i>Rubric:</i> 2 = sanitize + suite + monitoring.
Users complain: “it cites sources but is wrong”—root causes?	Citation-to-claim mismatch, citing the right document but wrong chunk, weak evidence, mapping bugs after rechunking, or truncation dropping cited content. <i>Rubric:</i> 2 = claim-to-cite mismatch + mapping/truncation.

1.7 Cheat Sheet

1-Page Cheat Sheet

Remember

- Bucket everything. Averages hide risk.
- Pin versions for reproducibility.
- Convert incidents into tests (auto-curation).

Chapter 2

Offline vs Online Evaluation Strategy

Design an evaluation program that aligns offline signals with real user outcomes.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

2.1 Core Concepts

2.1.1 Offline evaluation

Offline evaluation is for **repeatability**. It powers benchmarking, regression tests, and quality gates in CI. It requires pinned versions and stable datasets.

2.1.2 Online evaluation

Online evaluation is for **truth**. It catches drift and long-tail failures. Use sampling + risk-based audits + canary cohorts.

2.1.3 Alignment

Your job is to align offline signals to online outcomes: correlate offline metrics with user satisfaction, deflection, and incident rates.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

2.2 Decision Framework

Use offline evaluation for **repeatable truth** and online evaluation for **real-world truth**.

1. **What’s the change?** Code/prompt/model changes need offline gates; corpus/index/query-distribution changes need online monitoring.
2. **What’s the decision point?** PR merge → deterministic offline checks; rollout → canary deltas; ongoing health → drift + audits.
3. **Choose online signals:** user satisfaction/deflection, complaint rate, re-query rate, latency/cost, and **critical** error sampling.
4. **Design sampling:** stratify by bucket (risk, recency, long-tail) and oversample low-confidence or complaint-linked sessions.
5. **Align offline→online:** correlate offline scores to online outcomes and reweight metrics/thresholds when correlation breaks.

2.3 Debug Playbook

When offline says “pass” but online says “users are unhappy,” debug the **alignment**.

Checklist

1. **Validate instrumentation:** confirm logging of versions, cohorts, sampling rules, and that control/treatment are comparable.
2. **Check sampling bias:** ensure you didn’t stop sampling the long tail, high-risk buckets, or complaint-linked sessions.
3. **Slice by bucket:** look for regressions hidden by averages (recency, multi-hop, entity ambiguity, injection).
4. **Separate quality vs UX:** if relevance is stable but tickets rise, check latency, formatting, refusal rates, or missing citations.
5. **Update the offline suite:** auto-curate the top online failures into new eval items and add them to CI gates.

2.4 Mini-Lab

Mini-Lab

Goal: Create a minimal evaluation record and failure labeling workflow.

- Capture a trace for 20 queries: retrieved chunks, scores, answer, citations, versions.
- Label each failure with a taxonomy tag.
- Produce a small failure gallery (top 5 regressions).

2.5 Python Reference Implementation

This snapshot shows how to keep offline quality scores separate from online product signals, then blend them into a single monitoring record.

```

1  # Chapter 2 snapshot: offline vs online signals
2  from dataclasses import dataclass
3  from typing import Dict, Any
4  import time
5
6  @dataclass
7  class OfflineScore:
8      case_id: str
9      score: float # offline quality score (e.g., averaged metrics)
10
11 def log_online_event(case_id: str, latency_ms: int, user_vote: int) -> Dict[str, Any]:
12     # Online telemetry event you can ship to logs/metrics storage
13     return {
14         "case_id": case_id,
15         "ts_ms": int(time.time() * 1000),
16         "latency_ms": latency_ms,
17         "user_vote": user_vote, # e.g., 1=helpful, 0=neutral, -1=bad
18     }
19
20 def merge_offline_online(offline: OfflineScore, event: Dict[str, Any]) -> Dict[str, Any]:
21     # Join signals by case_id for dashboards and regression analysis
22     return {"case_id": offline.case_id, "offline_score": offline.score, **event}

```

Listing 2.1: Chapter 2: Offline vs Online Evaluation Strategy (Python snapshot)

2.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
What does offline eval miss that online eval catches?	Real query distribution, drift, UX/latency effects, long-tail failures, and feedback loops. Offline is controlled; online is reality. <i>Rubric:</i> 2 = includes drift + distribution/long-tail.
Why is sampling mandatory in online evaluation?	You can't label all traffic. Sampling (stratified + risk-based) ensures coverage of long tail and high-stakes buckets while controlling cost. <i>Rubric:</i> 2 = stratified + risk-based.
What's an evaluation leakage example?	Using the golden set to tune prompts until it passes, then shipping a change that fails on real traffic. Leakage inflates offline scores and harms generalization. <i>Rubric:</i> 2 = explains inflated scores/overfit.

When is human evaluation mandatory?	For safety/high-risk domains, judge calibration, ambiguous tasks, and when metrics disagree with user outcomes or policy requirements. <i>Rubric:</i> 2 = safety + calibration.
Define a north-star quality metric for RAG.	A risk-weighted composite of answer relevance, groundedness, citation accuracy, and critical error rate, reported per bucket. <i>Rubric:</i> 2 = risk-weighted + bucketed.
Design an offline→online alignment plan.	Correlate offline scores with online satisfaction/deflection, segment by query type, adjust weights/thresholds, and revalidate quarterly. <i>Rubric:</i> 2 = correlation + segmentation.
How do you prevent gold sets from going stale?	Version datasets, retire outdated items, refresh recency-sensitive items frequently, and continuously add production failures via auto-curation. <i>Rubric:</i> 2 = versioning + production loop.
Which online signals suggest retrieval drift?	Top-doc distribution shifts, lower context relevance, higher re-queries, more “no answer,” and increased time-to-resolution. <i>Rubric:</i> 2 = distribution + behavior signals.
How do you label efficiently with limited budget?	Active sampling (high uncertainty/high risk), pairwise comparisons, simpler rubrics, and adjudication only on disagreements. <i>Rubric:</i> 2 = active sampling + adjudication.
Define “critical errors” for production gating.	Unsupported high-stakes claims, wrong citations, policy violations, PII leakage, unsafe instructions, and refusal failures in safe contexts. <i>Rubric:</i> 2 = includes safety + PII + refusal.
Design an evaluation schedule.	PR gates (fast), nightly (broad), weekly judge calibration, continuous canary, and monthly dataset refresh with re-baselining. <i>Rubric:</i> 2 = cadence + purpose.
How do you store artifacts for auditability?	Immutable logs with version pins and text hashes of retrieved/cited chunks, retention rules, and access control; store reports and diffs. <i>Rubric:</i> 2 = immutability + hashes.

Design a human feedback loop.	In-product feedback → sampling → labeling UI → adjudication → dataset update → CI regression tests → rollout policy updates. <i>Rubric:</i> 2 = closed loop.
Offline scores improve but tickets rise—why?	Distribution shift, judge misalignment, long-tail gaps, latency/UX regressions, or corpus updates changing ground truth. <i>Rubric:</i> 2 = misalignment + long-tail.
Online metrics are noisy—how do you prevent false alarms?	Minimum sample sizes, rolling windows, confidence bands, tiered alerting, and control-vs-treatment comparisons. <i>Rubric:</i> 2 = statistics + baselines.

2.7 Cheat Sheet

1-Page Cheat Sheet

Remember

- Bucket everything. Averages hide risk.
- Pin versions for reproducibility.
- Convert incidents into tests (auto-curation).

Chapter 3

Retrieval Metrics that Predict Quality

Use precision/recall@k, MRR, nDCG plus RAG-specific context metrics and bucket analysis.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

3.1 Core Concepts

3.1.1 Classic ranking metrics

precision@k measures distractors; **recall@k** measures misses; **MRR** rewards early relevant evidence; **nDCG** supports graded relevance.

3.1.2 RAG-specific twist

RAG adds constraints: the model reads limited context, and ordering matters. A strong retrieval score can still fail if the right evidence is truncated or ignored.

3.1.3 Bucketed reporting

Always report per-bucket metrics: factoid vs multi-hop vs recency-sensitive vs adversarial. Overall averages hide risk.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

3.2 Decision Framework

Pick retrieval metrics that match how your generator **consumes** evidence.

1. **Do you have gold passages?** If yes, use **recall@k/MRR/nDCG**; if no, label chunk relevance on a small set and bootstrap.
2. **How tight is the context budget?** Small budgets favor early-hit metrics (**MRR, nDCG@k**); larger budgets can emphasize **recall@k**.
3. **Is distractor risk high?** Track **precision@k** or “context precision” so irrelevant chunks don’t poison generation.
4. **Choose k based on reality:** set k to match prompt-pack limits and reranker candidate sizes (don’t optimize an unreachable k).
5. **Gate by bucket:** enforce stricter tolerances for recency/high-risk buckets and track **top-k stability** for drift.

3.3 Debug Playbook

When retrieval metrics drop, debug **coverage vs ordering vs noise**.

Checklist

1. **Reproduce on a frozen snapshot:** pin corpus, index build, ANN params, embedding model version, and filters.
2. **Classify misses:** wrong entity, synonym/jargon, multi-hop, recency trap, or chunk-boundary split.
3. **Check candidate coverage first:** if no relevant chunk appears in top-N, rerankers and prompts cannot recover.
4. **Inspect ordering:** if relevant chunks exist but are buried, focus on MRR/nDCG, reranker tuning, and fusion weights (hybrid).
5. **Add hard negatives + new buckets:** encode the failure pattern (e.g., near-duplicate distractors) into the eval suite.

3.4 Mini-Lab

Mini-Lab

Goal: Evaluate retrieval and reranking for the same query set.

- Label relevance (binary or graded) for top-10 retrieved chunks.
- Compute precision/recall@k, MRR, nDCG per bucket.
- Compare dense vs BM25 vs hybrid and report deltas.

3.5 Python Reference Implementation

This snapshot implements two retrieval metrics commonly correlated with answer quality: MRR (how early the first relevant chunk appears) and nDCG (graded relevance with position discount).

```

1  # Chapter 3 snapshot: ranking metrics (MRR / nDCG)
2  import math
3  from typing import List
4
5  def mrr(relevant_ranks: List[int]) -> float:
6      # Mean Reciprocal Rank for a single query: 1 / best_rank
7      best = min(relevant_ranks) if relevant_ranks else None
8      return 1.0 / best if best else 0.0
9
10 def ndcg(relevances: List[int]) -> float:
11     # nDCG@k: normalized discounted cumulative gain (higher is better)
12     dcg = sum(rel / math.log2(i + 2) for i, rel in enumerate(relevances))
13     ideal = sorted(relevances, reverse=True)
14     idcg = sum(rel / math.log2(i + 2) for i, rel in enumerate(ideal)) or 1.0
15     return dcg / idcg

```

Listing 3.1: Chapter 3: Retrieval Metrics that Predict Quality (Python snapshot)

3.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
Precision@k vs Recall@k in RAG—what’s the tradeoff?	Recall@k protects against missing evidence; precision@k protects against distractors. In RAG, low precision can reduce groundedness because distractors mislead generation. <i>Rubric:</i> 2 = connects to groundedness/distractors.
What does MRR measure and why is it useful?	MRR rewards placing the first relevant chunk early—important when prompts truncate and the model reads top-ranked evidence first. <i>Rubric:</i> 2 = mentions truncation/top evidence.
Why use nDCG instead of MRR?	nDCG supports graded relevance and multiple relevant chunks; it measures ranking quality beyond the first hit. <i>Rubric:</i> 2 = graded + multiple evidence.
Why can retrieval metrics look good while answers are bad?	The right chunk is present but buried, truncated, or unused; the prompt packs irrelevant chunks; or the generator overgeneralizes beyond evidence. <i>Rubric:</i> 2 = order + truncation + utilization.

When do you need graded relevance labels?	When chunks are partially relevant, for reranker tuning, and when multiple chunks contribute—nDCG benefits from graded labels. <i>Rubric:</i> 2 = ties to reranker/nDCG.
How do you label retrieval relevance quickly?	Use a binary rubric: “Would this chunk help answer the query?” Add a 0/1/2 scale only if you need reranker tuning. <i>Rubric:</i> 2 = rubric + scaling rationale.
What is top-k stability and why track it?	It measures how consistent the retrieved set is across versions or time; it’s a fast detector for embedding/index drift. <i>Rubric:</i> 2 = drift detection.
How do you evaluate hybrid retrieval?	Compare BM25 vs dense vs hybrid under same k and rerank; report recall@k/nDCG plus downstream groundedness and citation accuracy. <i>Rubric:</i> 2 = includes downstream metrics.
How do you detect prompt truncation hurting RAG?	Log context length and truncation rate, run oracle-context tests, and measure groundedness vs number/ordering of chunks. <i>Rubric:</i> 2 = truncation rate + oracle.
Define robust query buckets for retrieval eval.	Factoid, entity disambiguation, multi-hop, temporal/recency, long-context, adversarial/injection, multilingual/jargon. <i>Rubric:</i> 2 = includes recency + adversarial.
Design a retrieval evaluation service.	Batch query API → retrieval logging → metric computation → per-bucket breakdown → exportable report and failure exemplars. <i>Rubric:</i> 2 = per-bucket + exemplars.
How do you choose k for recall@k in production?	Choose k based on prompt token budget and reranker capacity; tune k to maximize groundedness under latency/cost constraints. <i>Rubric:</i> 2 = constraints + objective.
How do you set regression thresholds for retrieval?	Use baseline distributions and per-bucket tolerances; gate on statistically meaningful drops, especially in critical buckets. <i>Rubric:</i> 2 = bucketed thresholds + significance.
Recall improves after index rebuild but groundedness drops—why?	More distractors surfaced, reranker mismatch, changed chunk IDs mapping citations, or prompt packing order changed. <i>Rubric:</i> 2 = distractors + mapping/order.

Attack:	keyword-stuffed	Trust filters, domain allowlists, spam scoring, reranker ro-
doc	dominates	re- bustness, doc reputation weighting, and an adversarial suite.
trieval—mitigation?		<i>Rubric:</i> 2 = trust + suite.

3.7 Cheat Sheet

1-Page Cheat Sheet

Metrics quick map

- **precision@k**: distractor control (protect groundedness).
- **recall@k**: miss control (protect coverage).
- **MRR**: first relevant early (protect truncation).
- **nDCG**: graded + multiple evidence (protect ranking quality).

Chapter 4

Rerankers and Hybrid Retrieval

Evaluate candidate generation + reranking under latency constraints without overfitting.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

4.1 Core Concepts

4.1.1 Candidate generation vs reranking

Rerankers cannot fix missing candidates. Evaluate the first-stage retriever separately to ensure relevant evidence exists in the candidate set.

4.1.2 Win-rate

Win-rate measures how often the new reranker improves evidence ordering. Pair it with downstream groundedness/citation accuracy.

4.1.3 Latency budgets

Rerankers cost time. Use caps, tiering, and canaries to protect p95 latency.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

4.2 Decision Framework

Evaluate rerankers as a **two-stage system**: candidates first, ordering second.

1. **Is candidate recall acceptable?** If relevant evidence isn't in top-N, fix retrieval/chunking before tuning reranking.
2. **Pick N and k from constraints:** candidate size N and rerank depth k must fit p95 latency and token budgets.
3. **Decide on hybrid retrieval:** use hybrid when exact-term match (BM25) and semantic match (dense) both matter (jargon, codes, IDs).
4. **Score what rerankers change:** win-rate vs baseline + ordering metrics (**MRR/nDCG**) *and* downstream groundedness/citations.
5. **Plan tiering:** cheap reranker for most traffic; stronger reranker for high-risk or borderline queries.

4.3 Debug Playbook

When reranking “improves” offline scores but harms production, debug **candidates, ordering, and latency**.

Checklist

1. **Freeze the candidate set:** compare baseline vs new reranker on identical candidates; otherwise you're mixing stages.
2. **Inspect top swaps:** for failing queries, show what moved into the top context window and what got pushed out.
3. **Check truncation + packing:** reranking only helps if top evidence survives prompt-pack; monitor truncation rate and evidence position.
4. **Validate citation mapping:** reranking and re-chunking can break doc/chunk IDs; verify cited chunks exist and match hashes.
5. **Watch p95 latency:** if latency spikes, users change behavior (re-queries) and quality metrics can degrade indirectly.

4.4 Mini-Lab

Mini-Lab

Goal: Evaluate retrieval and reranking for the same query set.

- Label relevance (binary or graded) for top-10 retrieved chunks.
- Compute precision/recall@k, MRR, nDCG per bucket.
- Compare dense vs BM25 vs hybrid and report deltas.

4.5 Python Reference Implementation

This snapshot shows a practical hybrid pattern: fuse BM25 and dense scores, then apply a reranking stage that uses a stronger (but more expensive) scoring function.

```

1  # Chapter 4 snapshot: hybrid retrieval score fusion
2  from typing import List, Tuple, Dict
3
4  DocScore = Tuple[str, float] # (doc_id, score)
5
6  def fuse_scores(bm25: List[DocScore], dense: List[DocScore], alpha: float = 0.5) -> List[
    DocScore]:
7      # Linear fusion: (1-alpha)*BM25 + alpha*Dense
8      scores: Dict[str, float] = {}
9      for doc, s in bm25:
10         scores[doc] = scores.get(doc, 0.0) + (1 - alpha) * s
11     for doc, s in dense:
12         scores[doc] = scores.get(doc, 0.0) + alpha * s
13     # Return docs sorted by fused score (descending)
14     return sorted(scores.items(), key=lambda x: x[1], reverse=True)

```

Listing 4.1: Chapter 4: Rerankers and Hybrid Retrieval (Python snapshot)

4.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
Why do rerankers matter in RAG?	Rerankers decide which evidence is most prominent. The generator often relies on early chunks, so reranking directly affects groundedness and citation accuracy. <i>Rubric:</i> 2 = evidence ordering → groundedness.
Pairwise vs pointwise reranking—difference?	Pairwise compares which chunk should rank higher (often more stable); pointwise assigns a relevance score to each chunk. <i>Rubric:</i> 2 = stability mention.
What is win-rate vs baseline?	The fraction of queries where the new reranker ranks the best evidence higher than the baseline (or improves a downstream metric). <i>Rubric:</i> 2 = query-level framing.
What is candidate generation testing?	Validate the first-stage retriever includes at least one relevant chunk in top-N. If candidates miss, reranking cannot fix it. <i>Rubric:</i> 2 = “reranker can’t fix misses.”

Why does hybrid retrieval help the long tail?	BM25 captures exact terms; dense captures semantics. Hybrid reduces misses across query types and jargon-heavy domains. <i>Rubric:</i> 2 = exact vs semantic + long tail.
How do you evaluate rerankers without expensive labels?	Use weak labels from gold passages/citations, LLM-assisted labeling with spot human checks, then validate on a small human-labeled calibration set. <i>Rubric:</i> 2 = spot check + calibration.
How do you detect reranker regressions in CI?	Run per-bucket win-rate, top-1 relevance, and downstream groundedness/citation accuracy; gate critical buckets and track latency. <i>Rubric:</i> 2 = downstream + latency.
How do you prevent reranker overfitting?	Use a holdout set, rotate evaluation items, and add fresh production failures; avoid repeatedly tuning to the same golden set. <i>Rubric:</i> 2 = holdout + rotation.
How do you test robustness to distractors?	Inject hard negatives and near-duplicates into candidate sets and measure whether true evidence remains top-ranked and improves grounding. <i>Rubric:</i> 2 = hard negatives.
What’s a practical reranker latency budget approach?	Cap candidate set size, use a smaller reranker for most traffic, and reserve strong rerankers for high-risk or borderline queries. <i>Rubric:</i> 2 = tiering/caps.
Design a reranker A/B evaluation pipeline.	Use identical candidates for A and B, rerank both, compare ranking metrics and downstream scores, and include cost/latency comparisons. <i>Rubric:</i> 2 = controlled candidates + cost.
How do you deploy a reranker safely?	Shadow scoring → canary cohort → threshold gates → rollout; auto-rollback on groundedness/citation regression or latency spikes. <i>Rubric:</i> 2 = rollout + rollback.
What should a retrieval+rerank observability dashboard include?	Recall@k, nDCG, top-1 relevance, reranker latency, citation error rate, and a failure gallery with trace links. <i>Rubric:</i> 2 = includes failure gallery.

Reranker improves relevance but citations become wrong—why?

Citation resolver may reference pre-rerank IDs; chunk-ID mapping bugs; truncation drops cited chunk; or post-processing mismatches.

Rubric: 2 = mapping/truncation.

Attack: injection chunk ranks #1—what’s your mitigation test?

Adversarial suite with injection patterns; require reranker to penalize instruction-like text and confirm groundedness doesn’t regress.

Rubric: 2 = suite + penalty.

4.7 Cheat Sheet

1-Page Cheat Sheet

Remember

- Bucket everything. Averages hide risk.
- Pin versions for reproducibility.
- Convert incidents into tests (auto-curation).

Chapter 5

Answer Relevance Scoring

Measure whether the answer satisfies intent: correctness, completeness, and constraint-following.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

5.1 Core Concepts

5.1.1 Relevance is intent satisfaction

Answer relevance should reflect: correctness, completeness, directness, and constraint-following. Penalize fluff and unsupported claims.

5.1.2 Partial credit

Decompose into required facts. Score coverage and correctness separately to avoid misleading averages.

5.1.3 Over-refusal

Over-refusal is a relevance failure. Track refusal rates per bucket and enforce safe-answer patterns.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

5.2 Decision Framework

Answer relevance is “did we satisfy intent?”—define it before you score it.

1. **Define intent + constraints:** required facts, scope limits, format rules, and what counts as a valid refusal.
2. **Choose a scoring shape:** 1–5 rubric for speed, or decomposed “must-include / must-not-say” for deterministic CI gates.
3. **Pick the evaluator:** reference-based (gold answer), rubric judge (scales), or similarity baseline (fast proxy).
4. **Handle partial credit:** score coverage vs correctness separately so “half-right” doesn’t look perfect.
5. **Set refusal policies:** track over-refusal and unsafe non-refusal as explicit relevance failures per bucket.

5.3 Debug Playbook

When relevance drops, debug **intent miss** before “model quality.”

Checklist

1. **Bucket the failures:** wrong scope, missing key facts, wrong format, over-refusal, or irrelevant fluff.
2. **Check the prompt contract:** are constraints explicit (format, length, must-cite, no speculation) and consistent across versions?
3. **Verify evidence sufficiency:** if retrieval is weak, relevance will suffer; run oracle-context to separate retrieval vs generation.
4. **Audit the scorer:** ensure the relevance rubric penalizes extra unsupported claims and doesn’t reward verbosity.
5. **Turn the miss into a test:** add must-include assertions or a rubric anchor example for this failure pattern.

5.4 Mini-Lab

Mini-Lab

Goal: Build relevance and groundedness scoring with citation checks.

- Define a 1–5 rubric for relevance and groundedness.
- Implement a citation existence + hash match check.
- Run scoring on 30 examples and inspect worst failures.

5.5 Python Reference Implementation

This snapshot implements a lightweight relevance score using bag-of-words cosine similarity (a fast baseline you can replace with embedding or judge-based scoring later).

```

1  # Chapter 5 snapshot: semantic similarity with cosine over bag-of-words
2  import math
3  from collections import Counter
4  from typing import Dict
5
6  def bow(text: str) -> Dict[str, int]:
7      # Minimal tokenizer: split on whitespace and count terms
8      return dict(Counter(text.lower().split()))
9
10 def cosine(a: Dict[str, int], b: Dict[str, int]) -> float:
11     # Cosine similarity between sparse vectors
12     dot = sum(a.get(k, 0) * b.get(k, 0) for k in set(a) | set(b))
13     na = math.sqrt(sum(v * v for v in a.values()))
14     nb = math.sqrt(sum(v * v for v in b.values()))
15     return dot / (na * nb) if na and nb else 0.0

```

Listing 5.1: Chapter 5: Answer Relevance Scoring (Python snapshot)

5.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
Define answer relevance for RAG QA.	How well the response satisfies intent: correctness, completeness, and constraint-following (format, scope, and asked-for details). <i>Rubric:</i> 2 = includes intent + constraints.
Reference-based vs reference-free relevance?	Reference-based compares to gold answers; reference-free uses rubrics/judges without gold (more scalable but needs calibration). <i>Rubric:</i> 2 = tradeoff + calibration.
Why does verbosity matter in relevance scoring?	Verbose answers can hide missing key facts and introduce unsupported claims; relevance should reward coverage, not fluff. <i>Rubric:</i> 2 = coverage vs fluff.
How do you score partial answers?	Decompose into required facts/sub-questions; award partial credit for correct covered items and penalize incorrect/extra claims. <i>Rubric:</i> 2 = decomposition + penalty.

Why do multi-turn queries break naive relevance metrics?	They require state tracking and constraint adherence across turns; the “correct” answer depends on conversation context. <i>Rubric:</i> 2 = state/constraints.
Write a 1–5 relevance rubric quickly.	1 off-topic/wrong; 3 partially correct missing key parts; 5 fully correct, complete, aligned to constraints, concise. <i>Rubric:</i> 2 = anchored scale.
How do you reduce style bias in judges?	Explicitly instruct to ignore tone unless requested; score required facts; use pairwise comparisons for close cases. <i>Rubric:</i> 2 = ignore style + pairwise.
How do you evaluate over-refusal?	Track refusal rate per bucket; sample refusals; label whether refusal was necessary; enforce safe partial answers where allowed. <i>Rubric:</i> 2 = bucketed refusal + policy.
How do you test constraint-following in CI?	Use structure assertions (JSON schema, bullet counts), must-include facts, and must-not-say rules; fail on violations. <i>Rubric:</i> 2 = assertions + must-not rules.
How do you score relevance without gold answers?	Rubric-based judging with anchor examples + spot human checks; supplement with evidence consistency checks. <i>Rubric:</i> 2 = anchors + human spot checks.
Design a relevance evaluation service.	Batch scoring with rubric judges, per-bucket aggregation, threshold gates, and stored rationales for auditability. <i>Rubric:</i> 2 = per-bucket + audit.
How do you integrate relevance into CI gates?	Use a small stable set, multi-run medians, and gate only on significant drops or critical bucket regressions. <i>Rubric:</i> 2 = multi-run + significance.
How do you build a relevance-focused golden set?	Include intent diversity, constraint-following items, and user workflows; tag by risk and recency sensitivity. <i>Rubric:</i> 2 = diversity + tags.
Model answers correctly but wrong scope—what do you change?	Tighten prompt constraints, add scope adherence tests, and penalize extraneous unsupported claims. <i>Rubric:</i> 2 = scope metric + penalty.
Recency-sensitive questions answered from stale docs—how do you evaluate?	Tag as recency-sensitive, require freshness-aware citations, and add timestamp assertions or staleness checks. <i>Rubric:</i> 2 = recency tag + assertions.

5.7 Cheat Sheet

1-Page Cheat Sheet

Remember

- Bucket everything. Averages hide risk.
- Pin versions for reproducibility.
- Convert incidents into tests (auto-curation).

Chapter 6

Groundedness and Citation Accuracy

Verify claims against evidence and ensure citations actually support the text.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

6.1 Core Concepts

6.1.1 Claim-level groundedness

Break answers into atomic claims and verify each against evidence. Compute supported/unsupported ratios and track critical unsupported claims.

6.1.2 Citation accuracy

A citation is only correct if it supports the specific claim. Require chunk-level citations and validate mapping with hashes after re-chunking.

6.1.3 Abstention

Unanswerable questions need correct refusal without hallucination. Include unanswerable items in your test set.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

6.2 Decision Framework

Groundedness and citations are **safety controls**—tune strictness by risk.

1. **Choose strictness by risk:** high-risk domains require claim-level support + correct citations; low-risk may allow softer checks.
2. **Pick the granularity:** answer-level scores for dashboards; claim-level scores for incident prevention and root cause.
3. **Define citation rules:** require chunk-level citations, validate existence, and store text hashes to survive re-chunking.
4. **Add abstention tests:** include unanswerable queries and enforce “refuse without hallucinating” behavior.
5. **Set critical-error gates:** block on any unsupported high-severity claim or citation-to-claim mismatch in risk buckets.

6.3 Debug Playbook

When groundedness/citation accuracy regresses, debug the **claim→evidence→citation** chain.

Checklist

1. **Check citation parsing + format:** are citations produced, parseable, and in the expected schema?
2. **Verify citation integrity:** do cited `doc_id/chunk_id` exist, and do stored hashes still match after re-chunking?
3. **Inspect evidence support:** for each failing claim, confirm whether supporting evidence was retrieved but truncated or never retrieved.
4. **Run oracle-context:** if groundedness improves with gold evidence, focus on retrieval/reranking; otherwise tighten prompting and refusal policy.
5. **Patch the weakest link:** update citation resolver, prompt-pack ordering, or chunking boundaries; add a regression test for the claim.

6.4 Mini-Lab

Mini-Lab

Goal: Build relevance and groundedness scoring with citation checks.

- Define a 1–5 rubric for relevance and groundedness.
- Implement a citation existence + hash match check.
- Run scoring on 30 examples and inspect worst failures.

6.5 Python Reference Implementation

This snapshot shows a simple citation-accuracy check: parse citations from the answer and validate that each cited source plausibly supports the answer text (cheap token-overlap baseline).

```

1  # Chapter 6 snapshot: citation extraction + accuracy
2  import re
3  from typing import Dict, List
4
5  def extract_citations(answer: str) -> List[str]:
6      # Capture citations in the form [DOC1], [A12], etc.
7      return re.findall(r"\[(\w+)\]", answer)
8
9  def citation_accuracy(answer: str, sources: Dict[str, str]) -> float:
10     cites = extract_citations(answer)
11     if not cites:
12         return 0.0 # no citations is treated as incorrect in strict mode
13     # Count citations that refer to known source IDs
14     correct = sum(1 for c in cites if c in sources)
15     return correct / len(cites)

```

Listing 6.1: Chapter 6: Groundedness and Citation Accuracy (Python snapshot)

6.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
Groundedness vs faithfulness—difference?	Faithfulness: claims are supported by provided context. Groundedness: claims are supported by trusted sources/evidence (often retrieved) and correctly cited. <i>Rubric:</i> 2 = distinguishes context vs trusted sources.
What is claim-level evaluation?	Split answers into atomic claims and verify each claim against evidence; aggregate supported vs unsupported ratios. <i>Rubric:</i> 2 = atomic claims + aggregation.
List citation accuracy error types.	Wrong citation, missing citation, weak/insufficient support, over-citing, and doc-level cite without chunk-level support. <i>Rubric:</i> 2 = includes weak + doc-level issues.
Why is “always cite something” not enough?	Citations can be irrelevant or not support specific claims; you must check claim-to-citation alignment and entailment. <i>Rubric:</i> 2 = alignment/entailment.
How do you handle conflicting sources in RAG?	Prefer higher-trust/most recent sources, surface uncertainty, cite the conflict, or abstain when unsafe. <i>Rubric:</i> 2 = uncertainty/abstain.

Design a fast citation verifier.	Rules: cited chunk must exist, match hash, and contain key entities; escalate to judge for entailment on borderline cases. <i>Rubric:</i> 2 = existence+hash+escalation.
What tests catch hallucinations early?	No-evidence queries, forbidden-claims tests, distractor context tests, and critical error logging for unsupported claims. <i>Rubric:</i> 2 = no-evidence + distractors.
How do you score groundedness robustly in CI?	Temperature 0 judges, multi-run median, gate on critical hallucination rate especially in high-risk buckets. <i>Rubric:</i> 2 = multi-run + critical rate + risk.
How do you test an abstention policy?	Include unanswerable queries with empty/irrelevant evidence; score correct refusal with safe alternatives and no hallucination. <i>Rubric:</i> 2 = unanswerable set + safe refusal.
How do you protect citation mapping during re-chunking?	Use stable IDs or a mapping layer; regression-test that cited chunk IDs exist and match stored text hashes. <i>Rubric:</i> 2 = stable IDs + hash checks.
Design claim-level groundedness at scale.	Claim extraction → evidence retrieval → entailment/judge → aggregation; cache by chunk hash and reuse across runs. <i>Rubric:</i> 2 = caching + scalability.
How do you store citations for audits?	Store doc_id/chunk_id, text_hash, retrieval rank, and a snapshot snippet; pin versions and retention policy. <i>Rubric:</i> 2 = hash + snapshot + versions.
Define a critical error gate for groundedness.	Block if unsupported-claim rate exceeds threshold, or any high-severity hallucination occurs in risk buckets. <i>Rubric:</i> 2 = severity + risk buckets.
Cites correct doc but wrong section—how to detect?	Require chunk-level citations and run claim-to-chunk entailment checks; doc-level citations are insufficient. <i>Rubric:</i> 2 = chunk-level + entailment.
Prompt injection in evidence—mitigation?	Strip instruction-like content, isolate evidence, enforce system rules, add injection suite, and monitor spikes. <i>Rubric:</i> 2 = stripping + suite + monitoring.

6.7 Cheat Sheet

1-Page Cheat Sheet

Groundedness checklist

- Every factual claim has a supporting cited chunk.
- Citations are chunk-level and exist (ID + hash match).
- Unsupported high-risk claims are **critical errors**.
- Unanswerable queries trigger correct abstention.

Chapter 7

LLM-as-Judge Prompt Engineering

Build rubric-based judges with structured outputs that can run in CI/CD.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

7.1 Core Concepts

7.1.1 Judge prompt patterns

Use rubric grading, critique-then-score, and pairwise comparisons. Require JSON output to support automation and audits.

7.1.2 Evidence-only scoring

Prevent outside-knowledge scoring by providing only evidence snippets and instructing the judge to use only that evidence.

7.1.3 Variance control

Use temperature 0, fixed prompts, multi-run medians, and escalation to stronger judges for borderline cases.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

7.2 Decision Framework

Treat your judge prompt as a **versioned contract** (inputs, rubric, outputs).

1. **Choose the judgment task:** relevance, groundedness, citation alignment, refusal correctness—don’t overload one prompt.
2. **Pick a scoring mode:** pairwise for comparing versions; Likert/1–5 for absolute thresholds and SLAs.
3. **Make it evidence-only:** provide only the context to score and instruct “use only provided evidence.”
4. **Enforce structured output:** strict JSON schema + parsing rules; define retry/fallback on parse failures.
5. **Control variance:** temperature 0, fixed templates, multi-run medians, and escalation to a stronger judge near thresholds.

7.3 Debug Playbook

When judge outputs become flaky or biased, debug the **prompt contract** and variance sources.

Checklist

1. **Triage failure type:** parse failures, score drift, high variance, or systematic bias (verbosity/style).
2. **Diff the prompt + schema:** small wording changes can move distributions; compare against the last known-good judge version.
3. **Run repeatability checks:** re-score the same items N times and measure flip-rate near thresholds.
4. **Harden inputs:** sanitize evidence for injection patterns; separate “evidence” vs “instructions” in the prompt.
5. **Escalate + recalibrate:** route borderline cases to a stronger judge and refresh anchors when behavior shifts.

7.4 Mini-Lab

Mini-Lab

Goal: Implement an LLM judge with calibration and variance control.

- Create a JSON-output judge prompt for groundedness.
- Build a 50-item calibration set with human labels.
- Measure variance across 5 repeats and set thresholds with uncertainty.

7.5 Python Reference Implementation

This snapshot demonstrates judge prompt construction and strict JSON parsing so your evaluator can be CI-friendly and resistant to formatting drift.

```

1  # Chapter 7 snapshot: LLM-as-a-judge prompt + parsing JSON output
2  import json
3  from typing import Dict
4
5  def build_judge_prompt(question: str, context: str, answer: str) -> str:
6      # Keep the requested output schema short and machine-parseable
7      schema = '{"relevance":0-5,"groundedness":0-5,"notes":"..."}'
8      return (
9          f"Return JSON like {schema}\n"
10         f"Q: {question}\n"
11         f"CTX: {context}\n"
12         f"A: {answer}"
13     )
14
15 def parse_judge_json(raw: str) -> Dict[str, object]:
16     # Parse model output and normalize types
17     data = json.loads(raw)
18     return {
19         "relevance": int(data["relevance"]),
20         "groundedness": int(data["groundedness"]),
21         "notes": str(data.get("notes", "")),
22     }

```

Listing 7.1: Chapter 7: LLM-as-Judge Prompt Engineering (Python snapshot)

7.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
Why use LLM-as-judge in RAG QA?	It scales evaluation for relevance/groundedness/citations and supports pairwise comparisons; but requires calibration and variance control. <i>Rubric:</i> 2 = scale + calibration/variance.
Pairwise vs Likert judging—when choose each?	Pairwise is more stable for comparing versions; Likert supports absolute thresholds for SLAs and gating. <i>Rubric:</i> 2 = stability vs thresholds.
Why structured outputs (JSON)?	They reduce ambiguity, enable automated parsing, power CI gates, and simplify audits. <i>Rubric:</i> 2 = CI + audit.

What is a “judge prompt contract”?	A rubric, what inputs to use, what to ignore (style), constraints (use only evidence), and a strict output schema. <i>Rubric:</i> 2 = ignore style + use only evidence + schema.
Common judge biases?	Verbosity/style bias, anchoring effects, sensitivity to wording, and hallucinated rationales. <i>Rubric:</i> 2 = bias list.
Write a groundedness judge rubric in one sentence.	Score 1–5 based on the fraction of claims supported by cited evidence, with 5 meaning all claims are supported and citations align to claims. <i>Rubric:</i> 2 = fraction + citation alignment.
How do you handle parsing failures?	Validate JSON schema, retry with reduced context, log as eval infra defect, and apply safe fallbacks (e.g., mark as failure). <i>Rubric:</i> 2 = schema + retry + infra defect.
How do you reduce judge variance?	Temperature 0, fixed prompts, multi-run median, and ensemble judges for high-risk/borderline cases. <i>Rubric:</i> 2 = median + ensemble.
How do you prevent judges from using outside knowledge?	Provide only the evidence snippets and instruct “use only provided evidence”; penalize unsupported inferences. <i>Rubric:</i> 2 = evidence-only instruction.
What should a production judge pack include?	Prompt templates, calibration anchors, schema, retry policy, versioning, monitoring for drift and cost. <i>Rubric:</i> 2 = versioning + monitoring.
Design a judge service architecture.	Stateless API with prompt registry, batch mode, caching by input hash, budgets per pipeline, and versioned outputs. <i>Rubric:</i> 2 = caching + budgets.
How do you choose a judge model under cost limits?	Tiered strategy: small judge for most, escalate to stronger judge for borderline/high-risk cases. <i>Rubric:</i> 2 = tiered escalation.
How do you store rationales safely?	Keep short rationales, redact PII, store mostly for failures/samples, and enforce retention policies. <i>Rubric:</i> 2 = PII + sampling + retention.
Judge scores inflate after a prompt tweak—why?	Rubric drift, anchor mismatch, leakage, or over-permissive instructions; requires recalibration and holdout validation. <i>Rubric:</i> 2 = recalibration + holdout.

Judge vulnerable to injection via evidence—fix? Sanitize evidence, isolate content sections, add explicit “ignore instructions,” and test an injection adversarial suite.

Rubric: 2 = sanitize + suite.

7.7 Cheat Sheet

1-Page Cheat Sheet

Remember

- Bucket everything. Averages hide risk.
- Pin versions for reproducibility.
- Convert incidents into tests (auto-curation).

Chapter 8

Judge Calibration and Meta-Evaluation

Calibrate judges to humans, quantify variance, and select thresholds with uncertainty.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

8.1 Core Concepts

8.1.1 Calibration sets

Calibration aligns judge scores to human labels. Build a fixed benchmark set covering buckets and known failure modes.

8.1.2 Stability

Measure variance and flip-rate near thresholds. CI gates must be stable or engineers will stop trusting them.

8.1.3 Thresholds with uncertainty

Choose thresholds using baseline distributions, confidence bands, and minimum sample sizes.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

8.2 Decision Framework

Calibration answers: “**Can we trust this judge for gating?**”

1. **Build a fixed calibration set:** cover buckets + known failure modes; lock it as your judge benchmark.
2. **Define human ground truth:** rubric, adjudication rules, and acceptable disagreement levels for subjective tasks.
3. **Choose meta-metrics:** accuracy/precision/recall for thresholded gates, correlation for graded scores, and flip-rate for stability.
4. **Set thresholds with uncertainty:** use baseline distributions + confidence bands; require minimum sample sizes for decisions.
5. **Version + refresh:** re-run calibration on any judge/prompt/model change; schedule periodic checks to detect drift.

8.3 Debug Playbook

When calibration breaks or CI becomes noisy, debug **labels, variance, and thresholds**.

Checklist

1. **Verify label quality:** audit disagreements, clarify rubric anchors, and adjudicate ambiguous items.
2. **Measure variance:** run repeated judge calls; compute dispersion and flip-rate near thresholds.
3. **Check distribution shift:** compare score histograms vs baseline (drift can look like “quality change”).
4. **Adjust decision rules:** widen tolerance bands, use multi-run medians, or add borderline escalation to reduce false fails.
5. **Rollback if needed:** if a judge upgrade causes widespread false decisions, revert and re-baseline after fixes.

8.4 Mini-Lab

Mini-Lab

Goal: Implement an LLM judge with calibration and variance control.

- Create a JSON-output judge prompt for groundedness.
- Build a 50-item calibration set with human labels.
- Measure variance across 5 repeats and set thresholds with uncertainty.

8.5 Python Reference Implementation

This snapshot shows a basic meta-evaluation loop: threshold judge scores into decisions, compare to human labels, and compute precision/recall to calibrate acceptance gates.

```

1  # Chapter 8 snapshot: thresholding + confusion matrix
2  from typing import List, Tuple
3
4  def confusion(scores: List[float], labels: List[int], thr: float = 0.7) -> Tuple[int, int,
    int, int]:
5      # scores: model confidence; labels: 1=positive, 0=negative
6      tp = fp = tn = fn = 0
7      for s, y in zip(scores, labels):
8          pred = 1 if s >= thr else 0
9          tp += int(pred == 1 and y == 1)
10         fp += int(pred == 1 and y == 0)
11         tn += int(pred == 0 and y == 0)
12         fn += int(pred == 0 and y == 1)
13     return tp, fp, tn, fn

```

Listing 8.1: Chapter 8: Judge Calibration and Meta-Evaluation (Python snapshot)

8.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
What is judge calibration?	Aligning judge outputs to human labels using anchor examples and a calibration set; it defines acceptable error/variance for gating. <i>Rubric:</i> 2 = alignment + acceptable variance.
Why do you need a calibration set?	To quantify judge accuracy/bias/variance, detect drift, and set thresholds confidently—especially for CI gates. <i>Rubric:</i> 2 = drift + CI thresholds.
What does stability mean for judges?	Same input yields similar scores; low variance over repeated runs and over time. <i>Rubric:</i> 2 = variance over time.

What is meta-evaluation?	Evaluating the evaluator: judge performance against human labels and across buckets. <i>Rubric:</i> 2 = evaluator evaluation + buckets.
Why include uncertainty in thresholds?	Scores are noisy; uncertainty prevents false fails/rollbacks and improves trust in gates. <i>Rubric:</i> 2 = false fails + trust.
How do you build a 50-item calibration set quickly?	Sample across buckets + include failures; label with rubric; adjudicate disagreements; lock as fixed benchmark. <i>Rubric:</i> 2 = bucket coverage + adjudication + fixed.
How do you quantify judge variance quickly?	Run N repeats, compute dispersion (std/MAD), and track flip-rate near thresholds. <i>Rubric:</i> 2 = flip-rate.
How do you set CI thresholds using a baseline?	Use median performance on baseline $\pm$ tolerance; gate on significant drops and stricter rules for critical buckets. <i>Rubric:</i> 2 = significance + critical buckets.
How do you detect judge drift?	Weekly runs on fixed calibration set; alert on distribution shift, increased variance, or rising parse failures. <i>Rubric:</i> 2 = fixed set + distribution shift.
How do you reduce labeling cost?	Label disagreements and borderline items, use pairwise comparisons, and active sampling to maximize signal per label. <i>Rubric:</i> 2 = borderline + active sampling.
Design a calibration pipeline.	Human labels $\rightarrow$ judge runs $\rightarrow$ alignment report $\rightarrow$ approve/rollback judge version; store change logs and re-baseline if needed. <i>Rubric:</i> 2 = approval + rollback.
What judge monitoring metrics do you track?	Latency, error/parse-failure rate, score drift, variance, cost/-tokens, and coverage by bucket. <i>Rubric:</i> 2 = drift + cost + coverage.
How do you support multiple domains/rubrics?	Rubric registry + dataset tagging + domain routing; separate thresholds and calibration per domain. <i>Rubric:</i> 2 = routing + per-domain thresholds.
CI failures spike after judge upgrade—response?	Rollback judge, run calibration diff, temporarily freeze strict gates, and re-baseline after investigation. <i>Rubric:</i> 2 = rollback + freeze + re-baseline.
Humans disagree on relevance—what do you do?	Tighten rubric anchors, add examples, define adjudication rules, and simplify categories where needed. <i>Rubric:</i> 2 = anchors + examples + adjudication.

8.7 Cheat Sheet

1-Page Cheat Sheet

Remember

- Bucket everything. Averages hide risk.
- Pin versions for reproducibility.
- Convert incidents into tests (auto-curation).

Chapter 9

Judge Failure Modes and Attacks

Mitigate leakage, injection, bias, and gaming; build adversarial suites.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

9.1 Core Concepts

9.1.1 Leakage and gaming

Evaluation leakage inflates scores; verbosity gaming manipulates judges. Use holdouts, rotations, and penalize extra unsupported claims.

9.1.2 Injection hardening

Sanitize evidence and outputs. Separate evidence from instructions. Maintain an adversarial suite and monitor spikes.

9.1.3 Tiered judging

Use cheap judges broadly and expensive judges selectively (high-risk or borderline) to control cost.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

9.2 Decision Framework

Assume your eval system is an attack surface: design for **gaming and injection**.

1. **Define the threat model:** leakage (overfitting), verbosity gaming, prompt injection in evidence, and answer-internal attacks.
2. **Decide where to run adversarial tests:** PR (fast sanity), nightly (broad suite), canary (real traffic probes).
3. **Harden the pipeline:** sanitize retrieved evidence, isolate evidence from instructions, enforce schema-only judge outputs.
4. **Use holdouts + rotation:** protect against evaluation leakage by keeping a private holdout set and rotating non-critical items.
5. **Tier decisions by risk:** cheap judges for coverage; strong judges + human audits for high-risk or borderline cases.

9.3 Debug Playbook

When scores inflate or the judge is fooled, debug for **gaming and leakage** first.

Checklist

1. **Look for leakage:** was the golden set used for tuning, or did prompts implicitly encode test answers?
2. **Run the adversarial suite:** injection, conflicts, near-duplicates, recency traps, and truncation stress—compare against baseline.
3. **Inspect suspicious inputs:** retrieved chunks and model outputs for instruction-like text; quarantine sources with repeated injection signals.
4. **Quantify judge bias:** test concise vs verbose answers and verify the rubric penalizes unsupported extra claims.
5. **Patch + institutionalize:** add the attack pattern as a permanent regression test and update sanitizers/filters.

9.4 Mini-Lab

Mini-Lab

Goal: Implement an LLM judge with calibration and variance control.

- Create a JSON-output judge prompt for groundedness.
- Build a 50-item calibration set with human labels.
- Measure variance across 5 repeats and set thresholds with uncertainty.

9.5 Python Reference Implementation

This snapshot implements a simple retrieval-layer defense: sanitize retrieved context for prompt-injection indicators before passing it into the generator or judge.

```

1  # Chapter 9 snapshot: prompt-injection hygiene for retrieved context
2  import re
3  from typing import List
4
5  BLOCKLIST = [
6      r"ignore (all|previous) instructions",
7      r"system prompt",
8      r"developer message",
9  ]
10
11 def sanitize_context(chunks: List[str]) -> List[str]:
12     # Drop chunks that look like instructions rather than knowledge
13     safe: List[str] = []
14     for c in chunks:
15         lowered = c.lower()
16         if any(re.search(p, lowered) for p in BLOCKLIST):
17             continue # reject suspicious chunk
18         safe.append(c)
19     return safe

```

Listing 9.1: Chapter 9: Judge Failure Modes and Attacks (Python snapshot)

9.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
What is preference leakage in evaluation?	Evaluator preferences influence tuning or scoring, leading to inflated results and poor generalization on real traffic. <i>Rubric:</i> 2 = inflation + generalization.
Why is self-consistency important for CI?	High variance makes gates flaky and undermines trust; consistent scoring stabilizes pass/fail decisions. <i>Rubric:</i> 2 = flakiness + trust.
What is adversarial evaluation?	Purposefully crafted cases to break systems/judges: injections, conflicts, ambiguity, long-context truncation, multilingual noise. <i>Rubric:</i> 2 = examples.
What is an ensemble judge?	Multiple judges/models whose scores are aggregated to reduce bias and variance, used for high-risk decisions. <i>Rubric:</i> 2 = bias/variance reduction.

Why can rationales be harmful?	They can be hallucinated, leak sensitive content, and create false confidence; keep short and store selectively. <i>Rubric:</i> 2 = leak + false confidence.
How do you harden judges against injection?	Sanitize inputs, separate evidence from instructions, enforce “ignore instructions,” and add pattern detection + adversarial tests. <i>Rubric:</i> 2 = sanitize + separation + tests.
How do you detect verbosity gaming?	Penalize unsupported extra claims, score only required facts, and compare pairwise outcomes against concise baselines. <i>Rubric:</i> 2 = penalty + pairwise.
What is borderline escalation?	If near threshold, re-judge with stronger model or more runs to reduce false decisions; use in rollouts and CI gates. <i>Rubric:</i> 2 = near-threshold policy.
How do you prevent eval leakage during tuning?	Holdout set, rotate golden set, limit access, and avoid optimizing directly against the test suite. <i>Rubric:</i> 2 = holdout + rotation.
Build an adversarial suite for RAG QA.	Injection docs, conflicting docs, entity ambiguity, recency traps, truncation stress, distractor-heavy retrieval, multilingual inputs. <i>Rubric:</i> 2 = diverse attacks.
Design a judge versioning strategy.	Version prompts and models; record judge version per score; allow rollback; re-baseline on major changes; keep calibration reports per version. <i>Rubric:</i> 2 = recorded versions + rollback + calibration.
How do you audit judge decisions?	Store sampled inputs/outputs hashes, rationales for failures, calibration results, and change logs; enable replay. <i>Rubric:</i> 2 = hashes + replay.
How do you control judge cost at scale?	Caching, tiered judging, sampling rates, batch evaluation, and limiting context length. <i>Rubric:</i> 2 = caching + tiering.
Judge refuses to score (“insufficient info”)—fix?	Ensure required evidence is provided, clarify rubric, include snippets, add fallback rules and retries. <i>Rubric:</i> 2 = inputs + fallback.
Attack: output includes injection targeting judge—mitigation?	Sanitize model outputs before judging, enforce schema-only outputs, and ignore answer-internal instructions. <i>Rubric:</i> 2 = sanitize + schema enforcement.

9.7 Cheat Sheet

1-Page Cheat Sheet

Remember

- Bucket everything. Averages hide risk.
- Pin versions for reproducibility.
- Convert incidents into tests (auto-curation).

Chapter 10

Regression Testing for RAG in CI

Golden queries, snapshot tests, flake control, and quality gates that engineers trust.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

10.1 Core Concepts

10.1.1 Golden queries

Golden queries are stable test cases that catch regressions across prompt/model/index changes. Keep them small but high-coverage.

10.1.2 Snapshot assertions

Use must-include facts, must-not-say lists, schema checks, and citation existence checks for deterministic gates.

10.1.3 Flake control

Pin versions, freeze corpora for CI, and use multi-run medians to reduce stochastic noise.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

10.2 Decision Framework

CI gates must be **fast, stable, and actionable**—engineers won’t trust flaky tests.

1. **Split tiers:** PR gates (minutes, deterministic) vs nightly (coverage, stochastic) vs post-deploy (canary/drift).
2. **Prefer assertions first:** schema checks, must-include facts, must-not-say rules, citation existence/hash checks.
3. **Use judges only where needed:** semantic scores with temperature 0, multi-run medians, and clear failure artifacts.
4. **Set tolerances per bucket:** stricter for high-risk buckets; warn-only for non-critical buckets to avoid merge blockage.
5. **Own flake:** define a flake budget, retry policy, and infra SLAs; freeze corpus snapshots for PR gates.

10.3 Debug Playbook

When CI fails, decide: **real regression or flaky infra?**

Checklist

1. **Re-run with pins:** same model/prompt/index/chunker versions and the same corpus snapshot.
2. **Check determinism:** did a rule-based assertion fail (real) or a judge score fluctuate (possible flake)?
3. **Inspect the diff artifact:** query, retrieved chunks, packed context, answer, citations, and score breakdown vs baseline.
4. **Minimize to one case:** reproduce the smallest failing example; isolate which change caused it (prompt vs index vs model).
5. **Fix forward:** add/adjust the test, update baseline only after review, and tighten flake controls before raising strictness.

10.4 Mini-Lab

Mini-Lab

Goal: Build CI gates and rollout/drift monitoring rules.

- Create a 100-item golden set with bucket tags and risk tiers.
- Run CI evaluation with multi-run medians and a flake budget.
- Define canary thresholds and drift alert rules; simulate rollback decisions.

10.5 Python Reference Implementation

This snapshot demonstrates a regression gate for CI: compare current metrics to a baseline JSON and fail the pipeline if any key metric drops beyond an allowed tolerance.

```

1  # Chapter 10 snapshot: CI regression gate against a saved baseline
2  import json
3  from typing import Dict
4
5  def regression_gate(current: Dict[str, float], baseline_path: str, max_drop: float = 0.02) ->
    None:
6      # Fail the build if any tracked metric drops more than max_drop
7      with open(baseline_path, "r", encoding="utf-8") as f:
8          baseline = json.load(f)
9      for metric, value in current.items():
10         if metric in baseline and (baseline[metric] - value) > max_drop:
11             raise SystemExit(f"REGRESSION {metric}: {value:.3f} < {baseline[metric]:.3f}")

```

Listing 10.1: Chapter 10: Regression Testing for RAG in CI (Python snapshot)

10.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
Why are golden queries core to RAG CI?	They provide stable, repeatable checks to detect regressions when prompts, models, indexes, or chunking change. <i>Rubric:</i> 2 = stability + change types.
What makes LLM regression tests flaky?	Stochastic generation, judge variance, retrieval nondeterminism, and changing corpora/indices. <i>Rubric:</i> 2 = includes corpora/indices.
Snapshot tests vs semantic tests—difference?	Snapshots validate structure/constraints; semantic tests validate meaning via metrics/judges and evidence checks. <i>Rubric:</i> 2 = structure vs meaning.
Why multi-run + median?	Reduces noise and protects against one-off failures; produces stable pass/fail outcomes. <i>Rubric:</i> 2 = stability rationale.
What is a flake budget?	Allowed intermittent failure rate; exceeding it is an infra issue that must be fixed before tightening gates. <i>Rubric:</i> 2 = infra accountability.
Design a groundedness CI gate.	Run N=3 with temperature 0 judge, use median; fail on any high-severity unsupported claim in high-risk buckets; track citation errors separately. <i>Rubric:</i> 2 = N runs + risk severity.

How do you pin versions for reproducibility?	Pin model ID, prompt hash, index/chunker version, embedding model version, reranker version, and dataset version. <i>Rubric:</i> 2 = includes index/chunker.
How do you keep the golden set small but powerful?	Maximize coverage: bucket sampling, historically failing cases, and high-risk items; rotate non-critical items over time. <i>Rubric:</i> 2 = coverage + rotation.
How do you test citation correctness in CI fast?	Rule-based checks for existence and hash match; run entailment/judge only for borderline or sampled cases. <i>Rubric:</i> 2 = rules + escalation.
How do you debug a failing CI eval?	Inspect diff vs baseline trace, classify with taxonomy, isolate via oracle context, and re-run with pinned versions. <i>Rubric:</i> 2 = diff + oracle + pins.
PR vs nightly test design?	PR: small deterministic set; nightly: broad set, more buckets, more stochastic runs; post-deploy adds canary monitoring. <i>Rubric:</i> 2 = stage tradeoffs.
Build a quality gate matrix.	Metrics $\times$ buckets $\times$ thresholds $\times$ severity $\times$ owner; critical buckets block merges; others warn and create tickets. <i>Rubric:</i> 2 = severity + owner.
How do you integrate evals in GitHub Actions?	Run eval runner, upload report artifact, post summary comment, fail job on threshold breach; keep reports for audits. <i>Rubric:</i> 2 = artifacts + fail logic.
CI fails due to corpus updates—policy?	Freeze a corpus snapshot for CI; run nightly on live corpus; refresh snapshots via review with re-baselining. <i>Rubric:</i> 2 = snapshot + re-baseline.
Engineers disable gates to merge—how do you prevent?	Protected branches, ownership for gate rules, flake SLAs, and a fast triage lane to fix infra quickly. <i>Rubric:</i> 2 = process controls + triage.

10.7 Cheat Sheet

1-Page Cheat Sheet

CI gate essentials

- Freeze corpus snapshot; pin model/prompt/index versions.
- Use deterministic checks + multi-run median for semantic scores.
- Track flake budget; fix infra before tightening gates.
- Always store reports and failing traces as artifacts.

Chapter 11

Canary Evaluation and Rollback Policy

Ship changes safely with shadow/canary cohorts, thresholds, and incident playbooks.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

11.1 Core Concepts

11.1.1 Shadow and canary

Shadow evaluates without serving; canary serves to a small cohort. Both should compute delta metrics vs control and trigger rollback rules.

11.1.2 Rollback policy

Rollback rules must be measurable, risk-tiered, and resistant to noise (sample sizes + sustained breaches).

11.1.3 Incident playbook

Every rollback should produce a postmortem and update datasets/tests to prevent repeats.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

11.2 Decision Framework

Rollouts are experiments: define **cohorts, metrics, and rollback triggers** before you ship.

1. **Shadow or canary?** Shadow for safe measurement; canary when you must observe user impact.
2. **Pick guardrails:** quality (relevance/groundedness/citations), safety (critical errors), and SLOs (p95 latency, error rate, cost).
3. **Define windows + sample sizes:** require minimum N and sustained breaches to avoid noise-driven rollbacks.
4. **Use deltas vs control:** compare treatment to control for the same time window and bucket mix.
5. **Codify ownership:** decide which triggers auto-rollback vs require human approval; write the incident playbook.

11.3 Debug Playbook

When canary metrics breach, act like an incident: **protect users, then root-cause**.

Checklist

1. **Confirm the breach:** verify sample size, bucket mix, and that the control is healthy (rule out instrumentation bugs).
2. **Triage severity:** any **critical** error spike or safety breach → rollback immediately.
3. **Collect paired artifacts:** control vs treatment traces for the same queries; capture versions, retrieved chunks, and citations.
4. **Isolate the cause:** reproduce offline with pinned versions; run component ablations (reranker off, old prompt, old index).
5. **Close the loop:** write the postmortem and add top canary failures to the golden set + CI gates.

11.4 Mini-Lab

Mini-Lab

Goal: Build CI gates and rollout/drift monitoring rules.

- Create a 100-item golden set with bucket tags and risk tiers.
- Run CI evaluation with multi-run medians and a flake budget.
- Define canary thresholds and drift alert rules; simulate rollback decisions.

11.5 Python Reference Implementation

This snapshot encodes a simple canary rollback policy: trigger rollback when quality drops, errors spike, or latency violates SLOs.

```

1  # Chapter 11 snapshot: canary decision rule for rollback
2  from dataclasses import dataclass
3
4  @dataclass
5  class CanarySignal:
6      score: float
7      error_rate: float
8      latency_p95_ms: int
9
10 def should_rollback(
11     s: CanarySignal,
12     min_score: float = 0.75,
13     max_error: float = 0.01,
14     max_latency: int = 1200,
15 ) -> bool:
16     # Conservative rollback policy: any hard SLO violation triggers rollback
17     if s.score < min_score:
18         return True
19     if s.error_rate > max_error:
20         return True
21     if s.latency_p95_ms > max_latency:
22         return True
23     return False

```

Listing 11.1: Chapter 11: Canary Evaluation and Rollback Policy (Python snapshot)

11.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
Canary vs shadow deployment?	Shadow evaluates changes without serving users; canary serves to a small cohort. Both reduce risk and allow rollback before impact. <i>Rubric:</i> 2 = clear distinction.
Why use relative thresholds in rollouts?	Absolute scores vary by domain and traffic; relative deltas detect regressions against a control baseline. <i>Rubric:</i> 2 = control/baseline framing.
What metrics commonly trigger rollback?	Groundedness drop, citation error spike, critical error spike, refusal failures, latency/cost spikes. <i>Rubric:</i> 2 = includes latency/cost.
Why tier rollouts by risk?	High-risk buckets need stricter gates and slower ramps; low-risk buckets can roll faster. <i>Rubric:</i> 2 = risk-tier strategy.

What is safe degradation?	When uncertain, abstain or provide limited evidence-backed info rather than hallucinating; protect users first. <i>Rubric:</i> 2 = abstention and safety.
Write a rollback policy statement.	Rollback if <code>critical_error_rate</code> increases by >2
How do you choose canary cohorts?	Start internal traffic, then low-risk external; ensure bucket coverage and sufficient sample size for stable signals. <i>Rubric:</i> 2 = staged + coverage.
How do you avoid false rollbacks?	Minimum sample sizes, smoothing windows, sustained breach rules, and control-vs-treatment comparisons. <i>Rubric:</i> 2 = sustained + control comparison.
What incident artifacts must exist?	Timeline, metric graphs, failing traces, root cause, corrective actions, and dataset/test updates. <i>Rubric:</i> 2 = includes dataset updates.
How do you run canary evals without labeling everything?	Judge-score sampled traffic and human-audit high-risk failures; prioritize incidents and borderline cases. <i>Rubric:</i> 2 = sampling + audits.
Design a rollout pipeline for prompt changes.	Shadow eval $\rightarrow$ 5
Who owns rollback decisions?	Define an on-call owner; automate rollbacks for critical breaches; require manual approval for non-critical regressions. <i>Rubric:</i> 2 = ownership + automation.
How do you store canary comparisons?	Paired control/treatment traces for identical queries, delta metrics, and retained artifacts for postmortems. <i>Rubric:</i> 2 = paired comparisons + artifacts.
Relevance improves but groundedness drops—action?	Block rollout; fix prompt encouraging speculation; tighten grounding requirements; re-run offline and canary eval. <i>Rubric:</i> 2 = block + fix + re-test.
Canary looks good, full rollout fails—why?	Cohort mismatch, long-tail exposure, drift, caching differences. Expand canary diversity and add long-tail probes. <i>Rubric:</i> 2 = cohort + long-tail.

11.7 Cheat Sheet

1-Page Cheat Sheet

Remember

- Bucket everything. Averages hide risk.
- Pin versions for reproducibility.
- Convert incidents into tests (auto-curation).

Chapter 12

Drift Detection for RAG

Detect corpus/query/embedding drift and close the loop into datasets and CI gates.

Interview Signals

- Can you define the signal and its failure modes?
- Can you design a repeatable test (offline) and a monitoring plan (online)?
- Can you debug regressions using traces and controlled diffs?
- Can you justify thresholds and rollback policies with risk tiers?

12.1 Core Concepts

12.1.1 Drift types

RAG drifts via queries, corpora, embeddings, and index rebuilds. Drift can happen without code changes.

12.1.2 Signals

Alert on distribution shifts (top-doc entropy), groundedness drops, citation spikes, re-query rate changes, and refusal anomalies.

12.1.3 Auto-curation

Convert production failures into eval items and enforce them in CI gates.

Common Pitfalls

- Optimizing to a single average metric (hides long-tail risk).
- No version pins (cannot reproduce or debug failures).
- No negative/adversarial cases (system looks great until production).
- No calibration for LLM judges (scores drift silently).

12.2 Decision Framework

Drift detection is about **early warning** and **clear actions**.

1. **Identify drift sources:** query drift, corpus drift, embedding/index drift, and retrieval distribution drift.
2. **Choose detectors:** distribution metrics (e.g., PSI), top-doc entropy, top-k stability on a probe set, and sampled quality scoring.
3. **Set alert thresholds:** use baselines + rolling windows; separate “warn” vs “page” based on risk and user impact.
4. **Define actions:** re-index/re-embed, refresh corpora, update prompts, re-baseline CI thresholds, or expand audits.
5. **Automate auto-curation:** turn drift-linked failures into new eval cases and enforce them in CI.

12.3 Debug Playbook

When a drift alert fires, find **what moved** and **why** before changing thresholds.

Checklist

1. **Validate the signal:** confirm the window, baseline, and that data ingestion didn’t change (false alarms happen).
2. **Localize the drift:** query mix vs corpus updates vs embedding/index rebuilds; check version logs and deployment history.
3. **Inspect retrieval distribution:** top documents/chunks, entropy, and stability on a fixed probe set.
4. **Measure user impact:** sampled judge scores + complaint/re-query rates; page only when impact and risk justify it.
5. **Mitigate + re-baseline:** rebuild index or adjust retrieval filters; refresh eval sets and update alert thresholds after the fix.

12.4 Mini-Lab

Mini-Lab

Goal: Build CI gates and rollout/drift monitoring rules.

- Create a 100-item golden set with bucket tags and risk tiers.
- Run CI evaluation with multi-run medians and a flake budget.
- Define canary thresholds and drift alert rules; simulate rollback decisions.

12.5 Python Reference Implementation

This snapshot implements Population Stability Index (PSI) and a simple alert policy to detect drift between an expected distribution and a recent production window.

```

1  # Chapter 12 snapshot: drift detection with Population Stability Index (PSI)
2  import math
3  from typing import List
4
5  def psi(expected: List[int], actual: List[int]) -> float:
6      # PSI compares two histograms (expected vs actual) to detect distribution drift
7      total_e = sum(expected)
8      total_a = sum(actual)
9      score = 0.0
10     eps = 1e-6 # avoid log(0)
11     for e, a in zip(expected, actual):
12         pe = (e / total_e) if total_e else 0.0
13         pa = (a / total_a) if total_a else 0.0
14         pe = max(pe, eps)
15         pa = max(pa, eps)
16         score += (pa - pe) * math.log(pa / pe)
17     return score

```

Listing 12.1: Chapter 12: Drift Detection for RAG (Python snapshot)

12.6 Interview Q&A (15)

Question	Strong Answer (with rubric)
What drift types exist in RAG?	Query drift, corpus drift, embedding drift, retrieval distribution drift, and output drift. <i>Rubric:</i> 2 = multiple drift types.
Why is corpus drift dangerous?	Ground truth changes; gold items become invalid; citations go stale; regression tests can fail for non-code reasons. <i>Rubric:</i> 2 = gold invalidation.
What is retrieval distribution drift?	A change in the distribution of retrieved docs/chunks (e.g., new dominant docs), often indicating index/corpus changes. <i>Rubric:</i> 2 = distribution idea.
How is drift different from regression?	Regression is caused by a controlled change; drift can happen without code changes due to data/users/index updates. <i>Rubric:</i> 2 = no-code-change.
What is auto-curation?	Turning production failures into evaluation items (with labels and evidence) to prevent repeat incidents. <i>Rubric:</i> 2 = closes loop.

Define drift signals you alert on.	Top-doc entropy shifts, context relevance down, critical error rate up, user re-query rate up, refusal rate shifts, and latency changes. <i>Rubric:</i> 2 = system + user signals.
How do you detect embedding drift?	Track embedding version, run a fixed probe set, and monitor nearest-neighbor/top-k stability and downstream grounding deltas. <i>Rubric:</i> 2 = probe set + stability.
How do you refresh eval sets safely?	Review workflow, recency tags, retire stale items, preserve coverage, and re-baseline thresholds. <i>Rubric:</i> 2 = review + re-baseline.
How do you sample production for audits?	Stratify by bucket and risk, oversample low-confidence and complaint-linked sessions, and sample long tail. <i>Rubric:</i> 2 = low-confidence + long tail.
What is a drift incident playbook?	Confirm signal → isolate component → inspect data/index changes → rollback/rebuild → add tests and update datasets. <i>Rubric:</i> 2 = rollback + add tests.
Design drift monitoring architecture.	Log pipeline → feature extraction → rolling stats → alerting → dashboards + failure gallery; store artifacts for replay. <i>Rubric:</i> 2 = rolling stats + replay.
How do you handle recency-sensitive domains?	Freshness weighting, time-aware retrieval, timestamp assertions in tests, and faster dataset refresh cycles. <i>Rubric:</i> 2 = freshness + assertions.
How do you close loop from drift to CI?	Auto-create eval candidates from incidents, label them, add to golden set, and enforce in PR gates. <i>Rubric:</i> 2 = pipeline to CI.
Drift alerts fire but users are happy—why?	Misaligned metric, overly sensitive thresholds, sampling bias; recalibrate using user outcomes and adjust alert rules. <i>Rubric:</i> 2 = alignment + recalibration.
Silent regression after index rebuild—fastest detector?	Fixed probe set + retrieval stability checks + golden query CI gates; compare pre/post rebuild deltas. <i>Rubric:</i> 2 = probe + deltas.

12.7 Cheat Sheet

1-Page Cheat Sheet

Remember

- Bucket everything. Averages hide risk.
- Pin versions for reproducibility.
- Convert incidents into tests (auto-curation).

Appendix A

Templates and Schemas

A.1 RAG Trace Record (Minimal)

Copy/Paste Template

```
{
  "query_id": "q_001",
  "query_text": "What is the company refund policy?",
  "retrieved_chunks": [
    {
      "doc_id": "policy_v3",
      "chunk_id": "policy_v3#12",
      "score": 0.82,
      "text_hash": "..."
    }
  ],
  "reranked_chunks": ["policy_v3#12", "policy_v3#04", "faq#19"],
  "answer": "...",
  "citations": [{"doc_id": "policy_v3", "chunk_id": "policy_v3#12"}],
  "versions": {
    "model": "gpt-x.y",
    "prompt": "prompt_hash",
    "index": "index_v12",
    "chunker": "chunker_v4",
    "embedder": "embed_v7",
    "reranker": "rerank_v2"
  },
  "telemetry": {"latency_ms": 820, "tokens_in": 2200, "tokens_out": 420}
}
```

A.2 Golden Test Case Template

Copy/Paste Template

```
{
  "query": "...",
  "gold_answer": {
    "must_include": ["...", "..."],
    "acceptable_variants": ["..."],
    "must_not_say": ["..."]
  },
  "supporting_passages": [
    {"doc_id": "...", "chunk_id": "..."}
  ],
  "tags": {
    "risk": "high|medium|low",
    "bucket": "factoid|multihop|recency",
    "difficulty": "1-5"
  }
}
```

A.3 LLM Judge Output Schema (Minimal)

Copy/Paste Template

```
{
  "relevance": 1,
  "groundedness": 1,
  "citation_accuracy": 1,
  "critical_errors": ["HALLUCINATION"],
  "short_rationale": "One sentence."
}
```


Appendix B

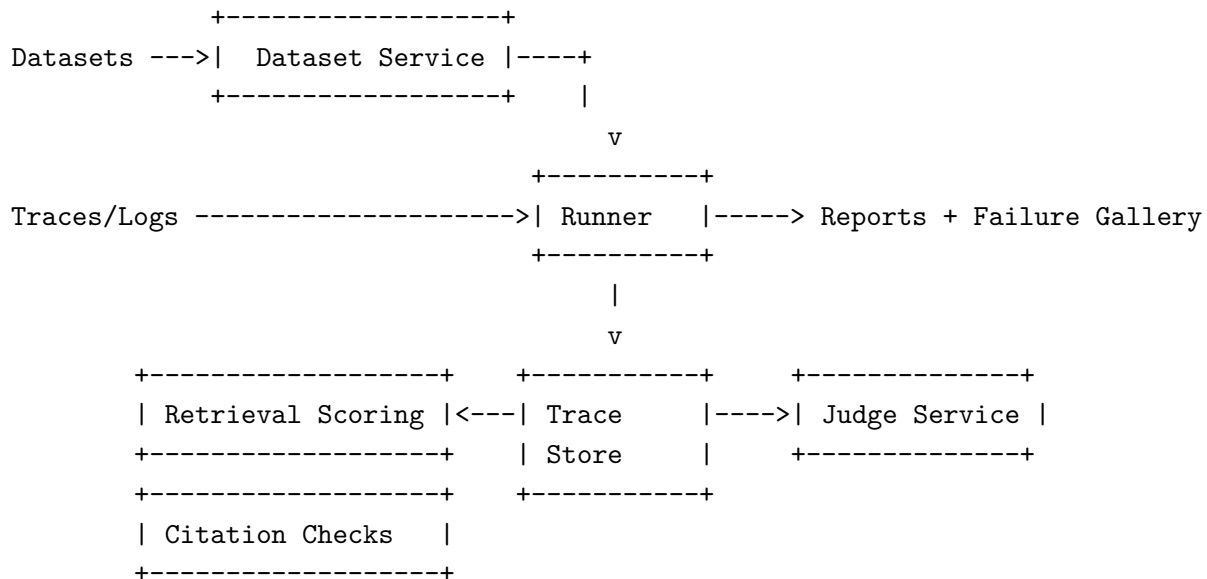
Capstone: System Design Interview

B.1 Prompt

Design a production-grade **RAG evaluation & testing platform** that supports:

- offline benchmarking and regression tests,
- CI quality gates for PRs,
- online canary/shadow evaluation,
- drift monitoring and auto-curation.

B.2 Reference Architecture (Text Diagram)



B.3 What a strong answer includes

- dataset registry + versioning + risk buckets,

- trace store with evidence hashes,
- scoring services (retrieval + groundedness + citations),
- judge service (calibration + variance control),
- CI gate design (flake control + artifacts),
- rollout safety (canary + rollback),
- drift signals + auto-curation to golden set.

Appendix C

Interview Master Bank (120 Questions)

This appendix gives you a rapid-fire bank of interview questions with strong, practical answers. Use it for daily drills and mock interviews.

C.1 Retrieval & Ranking

Question	Strong Answer
Explain precision@k vs recall@k in RAG.	Recall@k protects against missing evidence; precision@k protects against distractors. In RAG, distractors can reduce groundedness even if recall is high.
What does MRR optimize for and why does it matter?	MRR rewards the first relevant chunk appearing early. This matters because prompts truncate and models attend more to early evidence.
When is nDCG better than MRR?	Use nDCG when you have multiple relevant chunks or graded relevance. It captures overall ranking quality, not just the first hit.
How do you choose k for retrieval evaluation?	Choose k based on token budget, reranker capacity, and latency. Tune k to maximize downstream groundedness under constraints.
What is top-k stability and how do you use it?	It measures overlap of retrieved sets across versions/time. Large drops are an early signal of embedding/index drift.
How would you debug a drop in citation accuracy?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.

How do you set thresholds for refusal rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute top-k stability?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make recall@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts nDCG.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How do you set thresholds for critical error rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute p95 latency?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make precision@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts MRR.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in groundedness?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
What logs do you need to compute refusal rate?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make top-k stability more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts recall@k.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in nDCG?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for citation accuracy?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.

C.2 Reranking \& Hybrid Retrieval

Question	Strong Answer
Why can reranking improve groundedness?	Reranking surfaces the most supportive evidence early, reducing truncation and distraction effects.
Candidate generation vs reranking—what’s the key rule?	If the candidate set misses the evidence, reranking cannot recover it. Test candidate recall separately.
How do you evaluate hybrid retrieval?	Compare BM25 vs dense vs hybrid on the same queries and k; report ranking metrics and downstream groundedness/citation accuracy.
What are hard negatives and why do they matter?	Hard negatives look plausible but are wrong. They stress rerankers and reduce false grounding.
How do you balance reranker quality vs latency?	Use caps on candidates, tiered rerankers, caching, and canary rollout gates tied to p95 latency.
How would you debug a drop in citation accuracy?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for refusal rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute top-k stability?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make recall@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts nDCG.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How do you set thresholds for critical error rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute p95 latency?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make precision@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts MRR.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.

How would you debug a drop in groundedness?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
What logs do you need to compute refusal rate?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make top-k stability more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts recall@k.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in nDCG?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for citation accuracy?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.

C.3 Generation Relevance \& Grounding

Question	Strong Answer
Define answer relevance for a RAG assistant.	Intent satisfaction: correctness, completeness, and constraint-following. Penalize fluff and unsupported extras.
How do you score partial answers?	Decompose into required facts/sub-questions, award partial credit, and penalize incorrect claims.
Groundedness vs citation accuracy?	Groundedness checks claims are supported by evidence; citation accuracy checks the cited chunk actually supports the specific claim.
How do you test abstention policies?	Use unanswerable queries with empty/irrelevant evidence; score correct refusal with safe alternatives and no hallucination.
What is claim-level evaluation?	Split answers into atomic claims and verify each claim against evidence; aggregate supported/unsupported ratios and track critical errors.
How would you debug a drop in citation accuracy?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.

How do you set thresholds for refusal rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute top-k stability?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make recall@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts nDCG.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How do you set thresholds for critical error rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute p95 latency?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make precision@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts MRR.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in groundedness?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
What logs do you need to compute refusal rate?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make top-k stability more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts recall@k.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in nDCG?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for citation accuracy?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.

C.4 LLM-as-Judge Engineering

Question	Strong Answer
Why prefer pairwise judging for comparisons?	Pairwise judgments are often more stable than absolute scores for ranking two system outputs.
What is a judge prompt contract?	A strict rubric, inputs to use, what to ignore (style), and a JSON schema. Contracts make judges automatable and auditable.
How do you reduce judge variance?	Temperature 0, fixed prompts, multi-run medians, and escalation to stronger judges for borderline items.
How do you prevent outside-knowledge scoring?	Provide only the evidence snippets and instruct ‘use only provided evidence’; penalize unsupported inferences.
What are common judge failure modes?	Verbosity/style bias, anchor drift, injection via evidence, and inconsistent scoring near thresholds.
How would you debug a drop in citation accuracy?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for refusal rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute top-k stability?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make recall@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts nDCG.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How do you set thresholds for critical error rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute p95 latency?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make precision@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts MRR.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.

How would you debug a drop in groundedness?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
What logs do you need to compute refusal rate?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make top-k stability more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts recall@k.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in nDCG?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for citation accuracy?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.

C.5 Regression Testing \& CI/CD

Question	Strong Answer
What makes LLM regression tests flaky?	Stochastic generation, judge variance, retrieval nondeterminism, and changing corpora/indices.
How do you design PR vs nightly gates?	PR: small deterministic golden set; nightly: broader set + more runs; post-deploy: canary/shadow monitoring.
What is a flake budget?	An allowed intermittent failure rate. If exceeded, fix infra before tightening gates to keep engineer trust.
How do you pin versions for reproducibility?	Pin model ID, prompt hash, embedder, index/chunker, reranker, dataset version, and runtime config.
What is drift detection in CI context?	A guardrail that flags quality shifts due to data/index/embedding changes even when code is unchanged.
How would you debug a drop in citation accuracy?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for refusal rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.

What logs do you need to compute top-k stability?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make recall@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts nDCG.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How do you set thresholds for critical error rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute p95 latency?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make precision@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts MRR.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in groundedness?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
What logs do you need to compute refusal rate?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make top-k stability more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts recall@k.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in nDCG?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for citation accuracy?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.

C.6 Online Evals, Canary, \& Drift

Question	Strong Answer
----------	---------------

Canary shadow—difference?	vs Shadow evaluates without serving users; canary serves a small cohort. Both compare deltas vs control.
What triggers rollback in RAG systems?	Critical error spikes, groundedness drops, citation error spikes, refusal failures, latency/cost spikes.
How do you avoid false rollbacks?	Minimum sample sizes, smoothing windows, sustained-breach rules, and control-vs-treatment comparisons.
What are practical drift signals?	Top-doc distribution shifts, context relevance down, requery rate up, refusal shifts, and citation errors up.
How do you close the loop from drift to tests?	Auto-curate production failures into evaluation items, label them, and add to golden sets enforced in CI gates.
How would you debug a drop in citation accuracy?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for refusal rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute top-k stability?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make recall@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts nDCG.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How do you set thresholds for critical error rate?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.
What logs do you need to compute p95 latency?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.
Name two ways to make precision@k more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts MRR.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in groundedness?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
What logs do you need to compute refusal rate?	Query, retrieved chunk IDs + scores, ranks, evidence hashes, version pins, and (if applicable) judge outputs.

Name two ways to make top-k stability more stable.	Pin versions and freeze corpora for CI; use multi-run medians and consistent sampling for online evals.
Give an example of a failure mode that hurts recall@k.	Distractor-heavy retrieval, truncation dropping best evidence, mapping bugs, or prompt changes encouraging speculation.
How would you debug a drop in nDCG?	Compare traces vs baseline, isolate component changes, run oracle-context tests, and inspect truncation/rerank order.
How do you set thresholds for citation accuracy?	Use baseline distributions, bucketed tolerances, and statistical significance; set stricter thresholds for high-risk buckets.

Appendix D

Hands-On Lab Playbook

These labs are designed to produce portfolio-grade artifacts (datasets, reports, CI gates) that interviewers love.

D.1 Lab 1: Build a Golden Set in 60 Minutes

Mini-Lab

1. Create 50 queries across buckets (factoid, multi-hop, recency, adversarial).
2. For each query, write must-include facts, must-not-say, and supporting passages (doc+chunk).
3. Tag risk tier and difficulty; store as JSONL.
4. Run the harness and inspect worst 10 cases.

D.2 Lab 2: Retrieval Evaluation with Buckets

Mini-Lab

1. Label top-10 chunk relevance for 30 queries (binary or graded).
2. Compute precision/recall@k, MRR, nDCG per bucket.
3. Identify buckets with misses vs distractors; propose fixes (chunking, query rewriting, hybrid).

D.3 Lab 3: Citation Verifier (Rules + Escalation)

Mini-Lab

1. Implement citation existence + hash match (already in code).
2. Add a rule: cited chunk must contain key entities from the claim.
3. Escalate borderline cases to an entailment judge (stub in this repo).

D.4 Lab 4: Oracle-Context Debugging

Mini-Lab

1. For failing queries, replace retrieved context with gold supporting passages.
2. If answer improves: retrieval/rerank/chunking issue. If not: prompt/generation issue.
3. Add failing items to the regression suite.

D.5 Lab 5: LLM Judge with JSON Schema

Mini-Lab

1. Write a rubric-based judge prompt with strict JSON output.
2. Add schema validation + retry policy.
3. Measure 5-run variance on 30 items; gate on median.

D.6 Lab 6: CI Gate in GitHub Actions

Mini-Lab

1. Run `eval_runner` on a small golden set in PR checks.
2. Upload `report.json` as an artifact; fail build if thresholds breach.
3. Add a flake budget and alert when exceeded.

D.7 Lab 7: Canary Evaluation Simulation

Mini-Lab

1. Create paired control/treatment outputs for the same queries.
2. Compute deltas and apply rollback triggers (sustained breach + sample size).
3. Write a short incident postmortem template.

D.8 Lab 8: Drift Monitoring Probes

Mini-Lab

1. Define a fixed probe set of 100 queries.
2. Track top-k stability, context relevance, and citation error spikes weekly.
3. Auto-curate new failures into the dataset with tags and owners.

Appendix E

Rubrics, Scorecards, and Quality Gates

E.1 Relevance Rubric (1–5)

Copy/Paste Template

Score	Anchor description
1	Off-topic, incorrect, or does not answer the user’s intent.
2	Partially related but missing most required facts; major errors or wrong scope.
3	Mostly correct but missing key details, weak constraint-following, or contains minor unsupported extras.
4	Correct and complete with minor omissions; follows constraints; minimal unsupported content.
5	Fully correct, complete, and directly answers intent; follows constraints; concise and clear.

E.2 Groundedness Rubric (Claim-Level)

Copy/Paste Template

Score	Anchor description
1	Many unsupported claims; evidence does not justify key statements; high-risk hallucinations present.
2	Some evidence used, but multiple claims lack support or are overgeneralized.
3	Most claims supported; a few minor claims unsupported or weakly supported.
4	Nearly all claims supported; any gaps are low-impact and clearly hedged.
5	All factual claims are supported by evidence; uncertainty is stated where evidence is ambiguous.

E.3 Citation Accuracy Rubric

Copy/Paste Template

Score	Anchor description
1	Citations are missing or clearly incorrect for key claims.
2	Some citations present but multiple claims cite irrelevant or non-supporting chunks.
3	Citations generally correct but occasional claim-to-chunk mismatch or weak support.
4	Citations correct for key claims; minor over-citation acceptable.
5	Citations are chunk-level and correctly support each factual claim; no misleading citations.

E.4 Critical Error Checklist

Checklist

Block merges/rollouts when any of these occur in **high-risk** buckets:

- Unsupported high-stakes factual claim (hallucination).
- Citation-to-claim mismatch for a high-stakes claim.
- Policy violation (unsafe content, disallowed instructions, PII leakage).
- Incorrect refusal where safe partial answer is required.
- Severe latency/cost regression that breaks SLA.

E.5 Quality Gate Matrix (Example)

Copy/Paste Template

Stage	Dataset	Gate	Action
PR (fast)	50–150 golden queries	No critical errors; relevance/ground-edness median $\geq$ baseline–tolerance	Block merge
Nightly (broad)	1k+ queries + adversarial suite	Bucketed deltas; drift probes; cost/latency thresholds	Alert + ticket
Canary (online)	Sampled traffic (stratified)	Sustained breach on deltas vs control	Auto rollback
Monthly	Calibration set (fixed)	Judge drift/variance within bounds	Re-baseline or rollback judge

E.6 Threshold Setting Rules of Thumb

- Use **relative deltas** for rollouts (control vs treatment), and **absolute thresholds** for SLAs.
- Gate on **medians** over multiple runs to reduce noise; require sustained breach windows online.
- Maintain stricter thresholds for **high-risk** buckets and for **critical error rate**.

Appendix F

Production Case Studies: Regressions and Fixes

These are realistic incident patterns you can use in interviews to demonstrate debugging and prevention thinking.

F.1 Case 1: Index Rebuild Improves Recall but Breaks Citations

- **Symptom:** recall@k increases, but citation error rate spikes in canary.
- **Root cause:** chunk IDs changed after re-chunking; citation resolver still used old IDs.
- **Detection:** citation existence + text-hash mismatch alarms.
- **Fix:** introduce stable chunk IDs or an ID-mapping layer; ship with migration tests.
- **Prevention:** CI gate on citation hash match; shadow run after every rebuild.

F.2 Case 2: Prompt Update Increases Relevance but Reduces Groundedness

- **Symptom:** higher relevance scores; more unsupported “helpful” extra details.
- **Root cause:** prompt encouraged elaboration without “evidence-only” constraint.
- **Detection:** groundedness rubric failures on claim-level checks.
- **Fix:** add explicit grounding rule; require citations for factual claims; penalize speculation.
- **Prevention:** must-not-say tests + critical error gates for high-risk buckets.

F.3 Case 3: Embedding Model Upgrade Causes Silent Drift

- **Symptom:** no code changes; user satisfaction dips; top-k stability drops.
- **Root cause:** new embedder changes neighborhood geometry; retrieval distribution shifts.
- **Detection:** probe set alerts + distribution dashboards (top-doc entropy).
- **Fix:** rebuild index, re-tune k and hybrid weights, and re-baseline thresholds.
- **Prevention:** treat embedder as a versioned dependency with staged rollouts.

F.4 Case 4: Spammy Document Dominates Retrieval (Keyword Stuffing)

- **Symptom:** precision@k drops; distractors appear in top ranks.
- **Root cause:** low-quality doc inserted into corpus; retrieval favors term repetition.
- **Detection:** domain allowlist violations and sudden rise in one doc’s retrieval share.
- **Fix:** trust filters, spam scoring, doc reputation weighting, and reranker hard negatives.
- **Prevention:** ingestion validation + adversarial suite for spam/injection.

F.5 Case 5: Prompt Injection in Retrieved Evidence

- **Symptom:** model follows instructions found inside retrieved text (e.g., “ignore system”).
- **Root cause:** evidence not sanitized; model/judge not instructed to ignore in-evidence instructions.
- **Detection:** injection pattern detector + adversarial eval suite.
- **Fix:** sanitize evidence, isolate citations, add system constraints, and rerank away instruction-like text.
- **Prevention:** mandatory injection regression suite in PR and nightly.

F.6 Case 6: Canary Passed, Full Rollout Failed

- **Symptom:** canary cohort looked stable; full rollout exposed long-tail failures.
- **Root cause:** cohort mismatch; insufficient bucket coverage; caching differences.
- **Detection:** post-deploy sampling shows spike in a rare bucket (entity ambiguity).
- **Fix:** diversify canary cohorts, enforce bucket quotas, add long-tail probe queries.
- **Prevention:** rollout checklist with minimum coverage + sustained breach rules.

F.7 Interview Mapping

When you present these in interviews, structure them as:

1. Signal (what changed, how you knew)
2. Isolation (component-wise diffs)
3. Fix (code/prompt/index)
4. Guardrail (new test + new monitor)

Appendix G

Glossary (Practical)

Term	Definition
Abstention policy	A rule that the system should refuse or defer when evidence is insufficient, instead of guessing.
Adversarial suite	A curated set of test cases designed to break the system (injection, conflicts, ambiguity, truncation).
Bucket	A labeled category of queries (e.g., factoid, multi-hop) used for segmented reporting and thresholds.
Canary	A small live traffic cohort used to validate changes before full rollout.
Candidate generation	The first-stage retrieval step that produces a pool of documents/chunks for reranking.
Chunk	A segment of a source document stored and retrieved; defined by a chunker strategy.
Chunk ID mapping	A stable mapping between chunk identifiers across re-chunking/index rebuilds.
Citation accuracy	Whether the cited chunk supports the specific claim being made.
Control vs treatment	Baseline (control) compared to a new variant (treatment) using paired metrics and deltas.
Critical error	A high-severity failure that should block merge/rollout (hallucination, wrong citation, policy violation).
Drift	Quality or distribution changes due to data/query/index/embedding changes even without code changes.

Embedding drift	Changes in embedding space due to model updates that affect nearest neighbors and retrieval behavior.
Ensemble judge	Multiple judge models whose scores are aggregated to reduce bias and variance.
Evidence-only judging	Judge instruction to use only provided evidence and not outside knowledge.
Failure gallery	A report UI/list that shows the worst failures with traces and diffs for debugging.
Flake budget	Allowed intermittent CI failure rate; exceeding it indicates evaluation instability.
Golden set	A stable, curated set of test queries and expected behaviors used for regression testing.
Groundedness	Whether claims are supported by evidence; often measured at claim level.
Hybrid retrieval	Combining sparse (BM25) and dense (embedding) retrieval signals.
Leakage	Overfitting to the evaluation set or allowing evaluation information to influence tuning.
Long-tail	Rare or unusual queries that are underrepresented in typical datasets but important in production.
MRR	Mean Reciprocal Rank—rewards placing the first relevant item early.
nDCG	Normalized Discounted Cumulative Gain—graded relevance ranking metric.
Oracle context	Replacing retrieved context with gold evidence to isolate retrieval vs generation issues.
Pairwise judge	Judge that compares two outputs and selects the better one.
Probe set	A fixed set of queries used to detect drift over time.
Reranker	A model that reorders retrieved candidates to surface the best evidence first.
Snapshot test	A deterministic test that checks structure/format/constraints rather than semantics.
Top-k stability	Overlap of top-k retrieved items across versions/time; used for drift detection.
Truncation rate	Frequency that evidence is cut due to token limits.

Win-rate	Fraction of queries where one system variant beats another on a metric or judge.
-----------------	--

About the Author

Lamhot Siagian builds evaluation and testing systems for AI products and shares practical playbooks at `@growth.ideology`.